

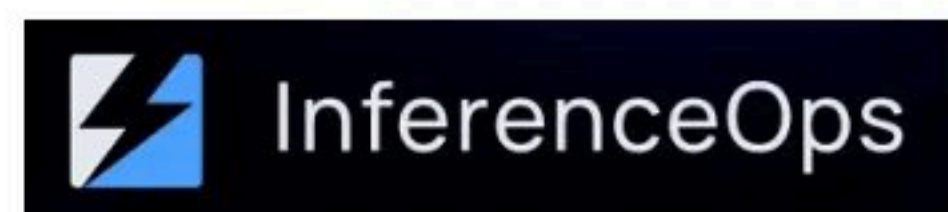


WELCOME

vLLM Inference Meetup

Bengaluru

19 September 2026



Agenda

Time	Session	Speaker
1:15–1:30 PM	Welcome Note	Prasad Mukhedkar
1:30–1:45 PM	Keynote 1: Sovereign AI Inference	Sudhir Dharanendraiah Senior Manager, Pyto
1:45–2:00 PM	Keynote 2: Tokenomics and Private AI Inference	Ramakrishna Yekulla
2:00–2:30 PM	Scaling Inference at NxtGen Using the vLLM ecosystem	Abhishek Kumar Singh
2:30–3:00 PM	Ilm-d for Sovereign & Agentic Workloads	Pravein Govindan Kannan
3:00–3:30 PM	Distributed Inference on ROCm WideEP on vLLM & Ilm-d	Chaitanya Sri Krishna Lolla & Sirra Ajith
3:30–4:00 PM	Coffee & snacks	
4:00–4:30 PM	Inside vLLM Semantic Router Intelligent routing for LLM inference	Aayush Saini
4:30–5:00 PM	Alligator.sh Powered by vLLM inference	Ompragash Prasad
5:00–6:00PM	Hands-on Workshop with Real GPU	AMD Team

From Serving Models to Scaling Models

SERVE

vLLM

Think Docker / Podman

Make one inference server extremely efficient.

- Continuous batching
- KV-cache management
- High GPU utilization

WHAT CHANGES AT SCALE?

100s/1000s GPUs
not just one server

Many replicas
where should each request go?

KV cache
can work be reused?

Prefill + decode
should they scale independently?



SCALE

llm-d

Think Kubernetes

Turn many inference servers into one system.

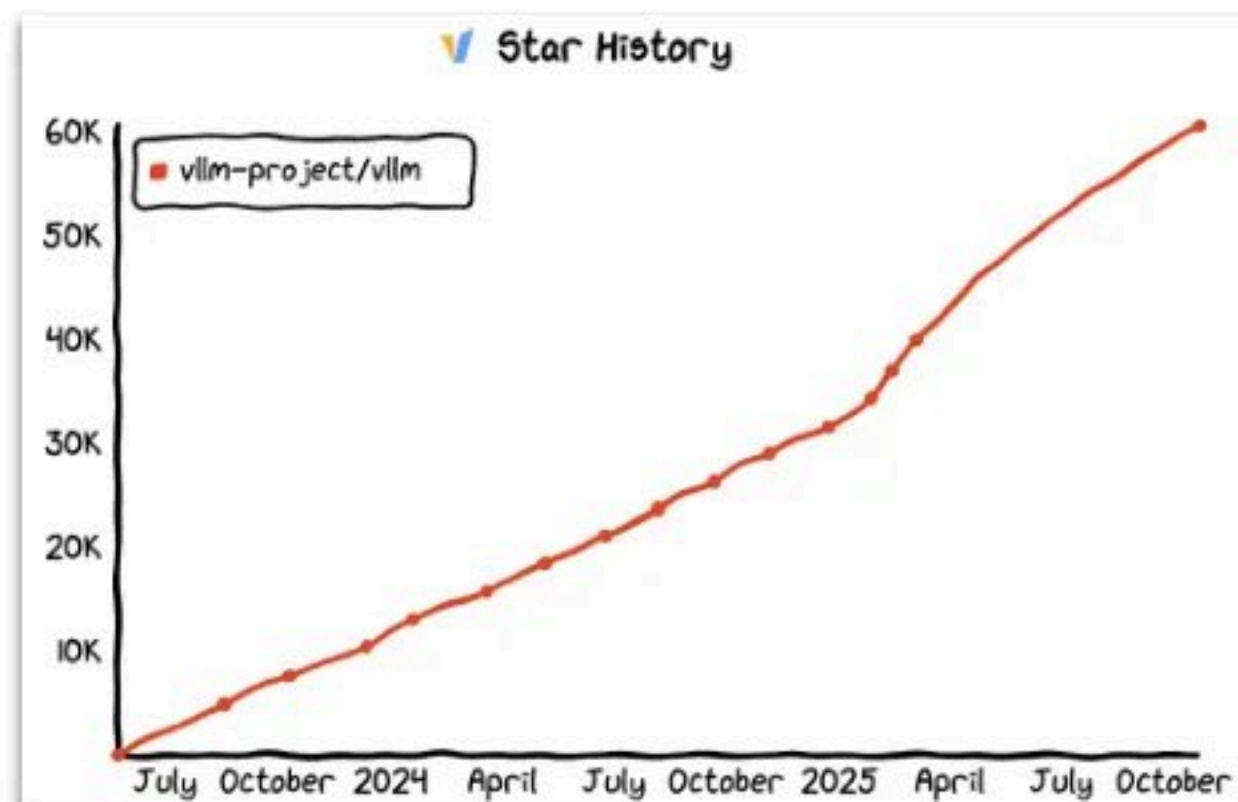
- Intelligent request routing
- Distributed / disaggregated serving
- Multi-node orchestration

The shift is not just more GPUs – it is from optimizing an engine to orchestrating a system.

vLLM makes each inference engine efficient. llm-d coordinates them at scale.



The High-Throughput and Memory-Efficient inference and serving engine for LLMs



Most Popular LLM Serving Engine

- PyTorch Foundation Hosted Project
- 89K+ GitHub stars, 800+ PRs/month
- 500K++ GPUs deployed 24/7
- 3K+ contributors, 10K+ members in slack.vllm.ai
-



<https://github.com/vllm-project/vllm>

```
$ uv pip install vllm --torch-backend=auto
$ vllm serve deepseek-ai/DeepSeek-V3.1 -tp 8
```

Broad Model Support (>100 arches)



Flexible Device Parallelism

Tensor, Pipeline, Expert, Data, Context
Parallel

Disagg Prefill/Decode, Disagg Encoder

Wide Hardware Support



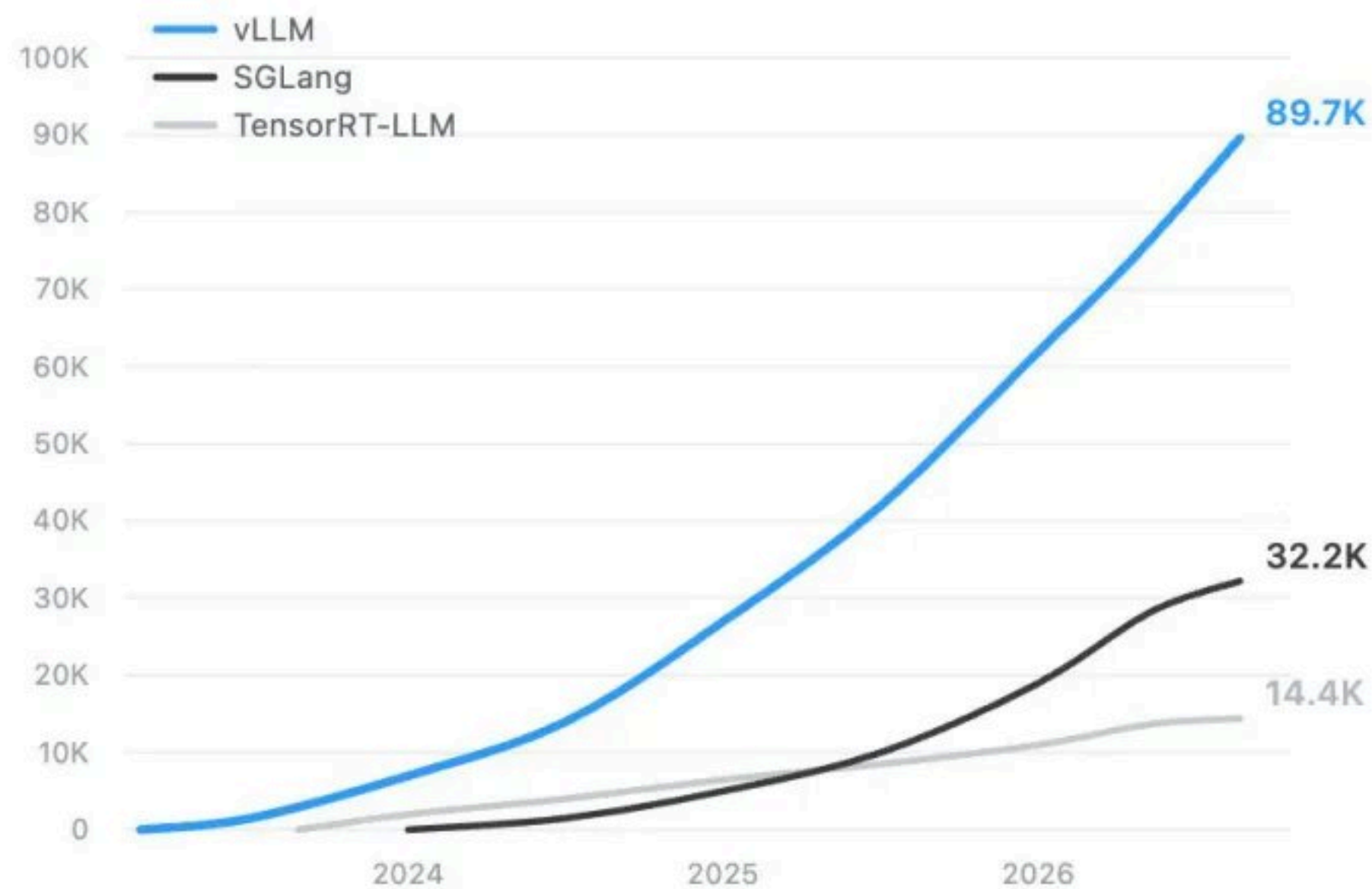
Diverse Project Ecosystem



The state of vLLM today

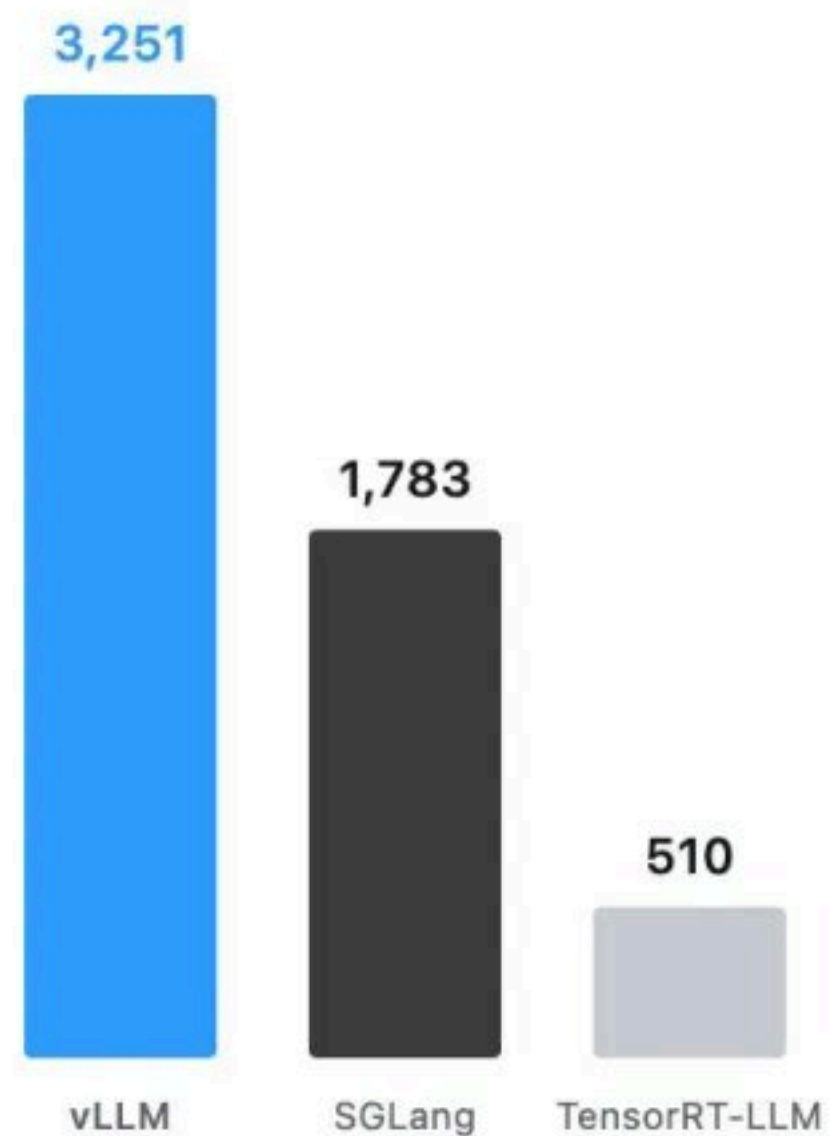
vLLM's growth as the center of open source inference

GitHub stars over time



Source: GitHub, August 2026

Contributors



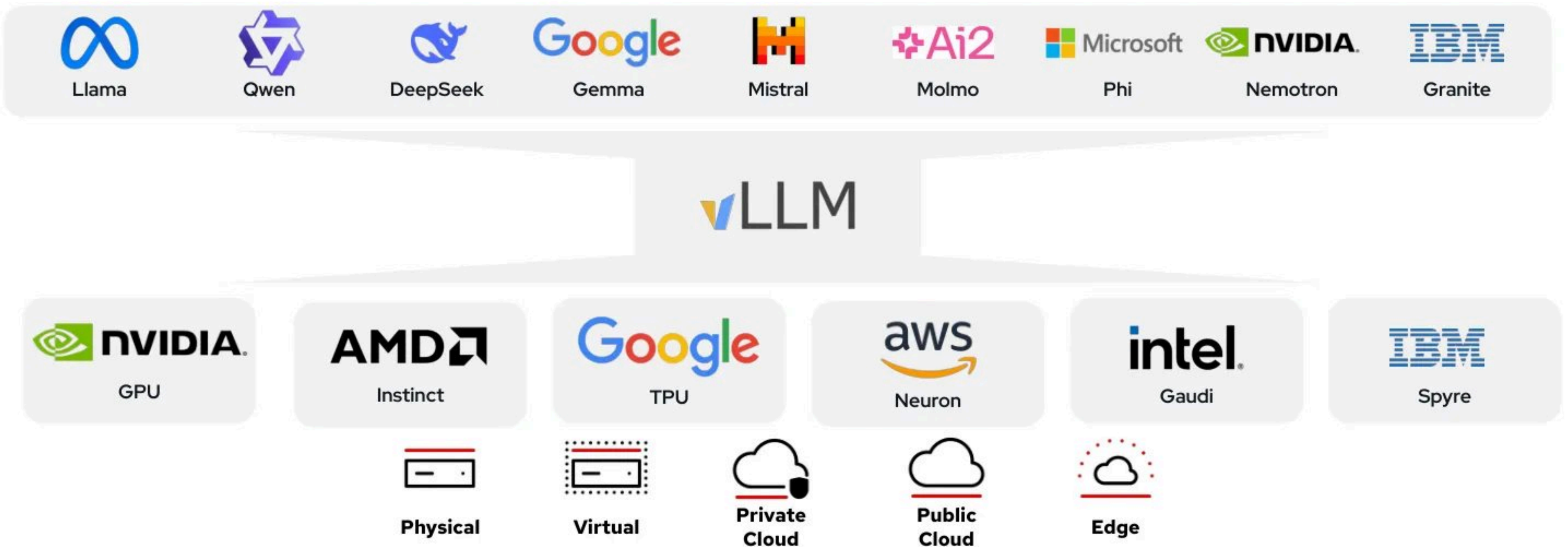
89.7K

GitHub stars

3,251

contributors and counting

vLLM connects model creators to accelerated hardware providers



Bookmark1: https://recipes.vllm.ai/

 / Recipes

[Browse](#) [Docs](#) [GitHub](#)

vLLM Recipes

Pick a model, adjust for your GPUs, copy the `vllm serve` line that runs. Community-maintained recipes for NVIDIA H100/H200/B200/B300, Grace-Blackwell, and AMD MI300X/MI325X/MI355X. [Full vLLM compatibility](#)

LATEST RECIPES

newest 8

View all 191

**Intern-S2-397B**
internlm

@ 397B/17B

BF16

262K ctx

Multimodal

T Text

Flagship scientific multimodal MoE (397B total / 17B active) on the Qwen3.5 hybrid linear/full-attention...

**Qwen3.8-Flash-Next**
Qwen

@ 176B/68

NVFP4

262K ctx

Multimodal

T Text

Qwen4 architecture preview with a 125B-parameter main model, supplemented by an additional 51B N...

**DeepSeek-V4.1-Flash**
deepseek-ai

@ 552B/8-16B

FP8

1M ctx

T Text

Multimodal

DeepSeek V4.1 Flash vision-language MoE (552B backbone; 8B active per prompt token, 16B per outpu...

**Ling-3.0-flash**
inclusionAI

@ 124B/5.5B

BF16

262K ctx

T Text

Ling-3.0-flash MoE model with BF16, FP8, FP4, and INT4 checkpoints, native MTP, and an external DSpar...

**Ling-3.0-flash-VL**
inclusionAI

@ 124.85B/5.51B

BF16

131K ctx

T Text

Multimodal

Ling-3.0-flash vision-language MoE model with BF16, FP8, FP4, and INT4 checkpoints

**MiniCPM5-2B**
openbmb

@ 2B

BF16

131K ctx

T Text

MiniCPM5-2B — dense 2B LLM with hybrid Think/No-Think reasoning, native 128K context, and native tool...

**DeepSeek-V4-Flash-Vision-Exp**
deepseek-ai

@ 285B/13B

FP8

1M ctx

T Text

Multimodal

DeepSeek's first experimental multimodal V4 model — the V4-Flash MoE backbone plus a 32-layer vision...

**K2-Horizon-0.9B**
IFM

@ 1.08B

BF16

131K ctx

T Text

Compact dense model for mathematics, code, instruction following, and STEM tasks

BROWSE BY PROVIDER

**Arcee AI**
arcee-ai
1 recipe

**Ernie (Baidu)**
baidu
3 recipes

**Boson AI**
bosonai
1 recipe

**Seed (ByteDance)**
ByteDance-Seed
1 recipe

**DeepSeek**
deepseek-ai
11 recipes

**Dots**
dots-studio
1 recipe

Bookmark2: <https://llm-d.ai/docs/well-lit-paths>

Tested, benchmarked recipes for running LLM inference workloads in production

Intelligent Inference Scheduling

KV cache locality estimation routing, load-aware routing

"Precise" prefix aware routing

prefix match against KVEvents API routing

Prefill-Decode Disaggregation

Independent pools per inference phase

KV Cache Management

KV-offloading, P2P KV-sharing

Batch Serving

Maximize throughput for offline inference

Multi-Node "Wide Expert Parallel" Serving

DP attention + EP MoE across nodes

Flow Control

Multi-tenancy, fairness-scheduling

Workload Variant Autoscaling

Scale variants to workload demand

Reinforcement learning

Scale rollout generation without slowing training

Multimodal Serving

Preserve reusable reasoning context across image and audio generation

Agentic Serving – workload-centric

Start from the workload; compose the capabilities it needs per accelerator



KEYNOTE 01

Sovereign AI Inference

Sudhir Dharanendraiah

Red Hat

A simple question

Where does your AI actually live?

**If inference leaves your control,
can you really call the AI sovereign?**

The model is only one layer of the system.

Sovereignty is a systems property

03

It is not a checkbox on the model card.

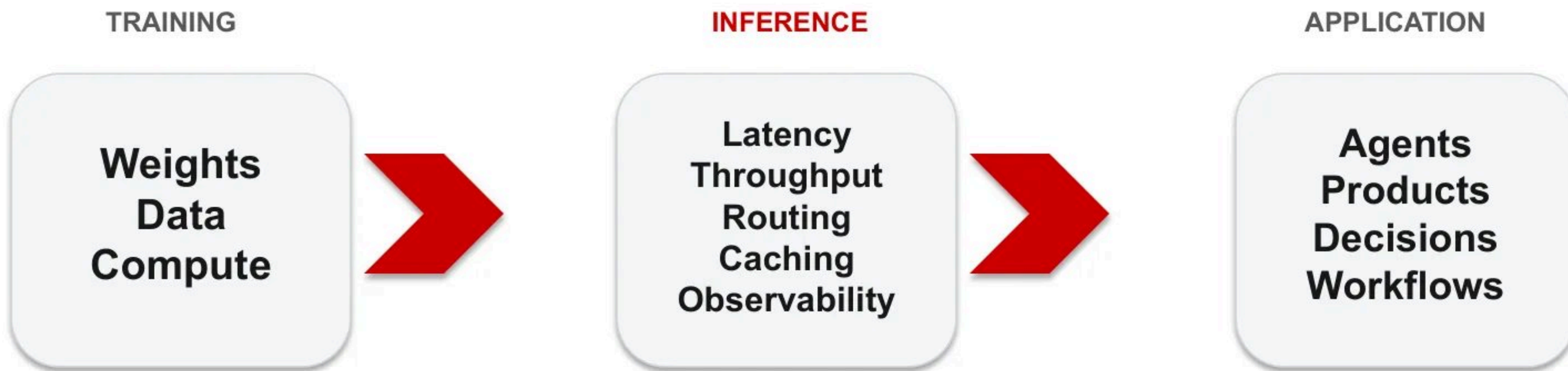


Sovereignty = freedom to choose, control, verify and operate.

Inference changes the sovereignty equation

04

Training creates capability. Inference turns it into an operating system for intelligence.



The strategic question becomes: who controls the serving layer?

The open inference stack

05

A sovereign stack is not one product. It is a set of interoperable layers.

APPLICATIONS & AGENTS	Where intelligence creates value
DISTRIBUTED INFERENCE	Routing · scheduling · autoscaling · KV-cache awareness
INFERENCE ENGINE	vLLM · kernels · compilation · quantization
MODEL & ARTIFACTS	Open models · weights · provenance · policy
ACCELERATORS	NVIDIA · AMD · Intel · other platforms
FOUNDATION	Linux · Kubernetes · observability · security

vLLM makes inference fast. The next challenge is making it sovereign at scale.

06

Think engine + distributed operating layer.

vLLM

- High-throughput serving
- PagedAttention + KV-cache efficiency
- Continuous batching
- Compilation + optimized kernels
- Distributed execution

llm-d

- Kubernetes-native distributed inference
- Intelligent routing
- Autoscaling + resource efficiency
- Observability
- Production-scale orchestration

Open inference infrastructure becomes a strategic capability.

Sovereignty requires hardware freedom

07

A sovereign architecture should not turn into a hardware lock-in.



Choice is not an implementation detail. It is part of sovereignty.

The goal is not to eliminate every dependency — it is to avoid dependencies you cannot change.

Sovereign inference must be verifiable

08

Fast inference is not enough.

PROVENANCE

What model, code and artifacts entered production?

REPRODUCIBILITY

Can we recreate the serving environment?

SECURITY

Can we inspect, patch and validate the stack?

OBSERVABILITY

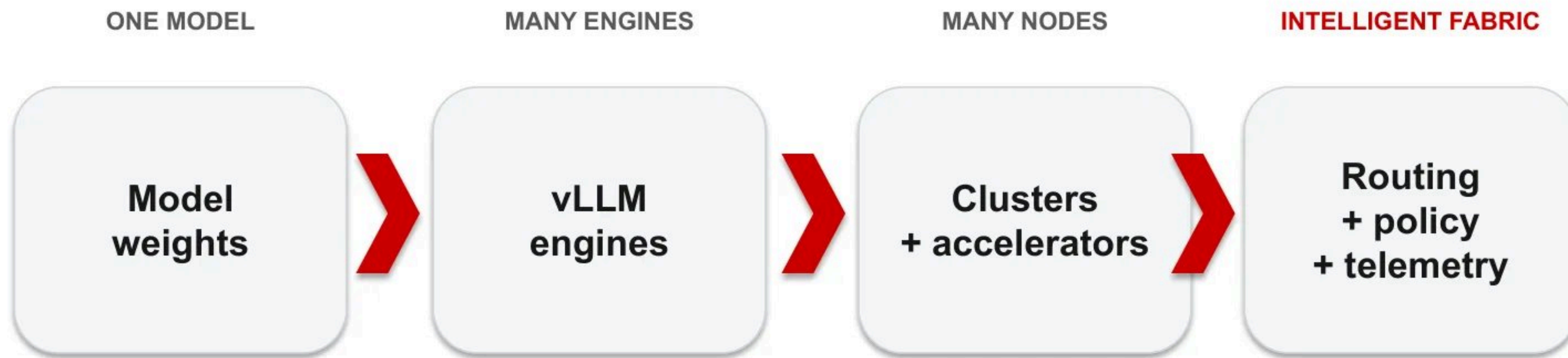
Can we see performance, failures and resource use?

Trust comes from the ability to inspect and reproduce — not simply from where the software came from.

The next frontier: distributed intelligence

09

Inference is becoming a distributed system.



The opportunity is to make intelligence portable, composable and locally controllable.

Sovereignty is not about building everything yourself.

**It is about having the freedom
to choose, control, verify and operate.**

**Open source gives us the foundation.
Open inference gives us the leverage.
Engineering gives us the sovereignty.**

Thank you.

KEYNOTE 02

Tokenomics and Private AI Inference

Ramakrishna Yekulla

Red Hat



Quick show of Hands.

The Token Raj :

How frontier AI vendors
colonized your inference stack



The Subsidy Is Over

Frontier labs kept token prices unsustainably low to grab market share. That window has closed. Capex is at record highs, and the bill is coming — **you're the one paying it.**

\$12/M

Google Gemini

Triples in price each year. Aggressive land-grab pricing now reversing.

\$25/M

Anthropic Claude

Premium pricing maintained. Recent cuts only on top-tier models.

\$30/M

OpenAI GPT-4o

Flagship was subsidized, then flipped upward as margins tightened.

📌 **"Cuts when cash is plentiful & share matters. Increases when cash is tight & margins matter."** — The latter is now the case for all three vendors simultaneously.

Your Tokens Are Shrinking

Bigspin built a **Consumer Price Index for AI coding output** — tracking 5,851 real Opus 4.6 sessions over 10 weeks. Same model. Same tasks. A token in mid-April bought just **\$0.23 on the February dollar** — a 4.3× cost increase with zero warning.

Where Did the Tokens Go?

In February, nearly every output token wrote code. By April, **thinking rose from 1% → 18%**, and explanation ate the rest. You're being billed for the model's internal monologue — the AI equivalent of a consultant billing you for thinking about billing you.

What 10,000 Tokens Bought You

Metric	Feb	Apr	Decline
PRs shipped	0.30	0.16	47% gone
Code lines drafted	630	91	86% gone
Files touched	8.2	3.6	56% gone
Knowledge captures	0.95	0.42	56% gone

Forecasts that hold tokens-per-output constant will **drastically underestimate your spend**.

The Token Industrial Complex

Vendor revenue scales with **tokens burned**, not value created — a structural incentive to make models chatter longer, reason further, and generate more. During the Bigspin study, token usage **spiked across every user cohort** for a full week mid-study. No documented feature change. No session-length shift. No explanation. Even the researchers who built the CPI couldn't explain their own bill.

The Black Box Problem

If the meter can move and careful outside observers can't tell you why, ask yourself how much visibility *you* have inside someone else's black box.

Diminishing Returns Are Real

Doubling your token spend **does not double your accuracy**. Test-time scaling research confirms diminishing returns hit fast — more tokens ≠ better output.

The Fix: Token Leverage

Stop optimizing for token burn. Start optimizing for **token leverage** — smaller, tuned, task-specific models where they fit. Route the rest elsewhere.

The Sovereignty Illusion

We encrypt data **at rest** and **in transit** — but during inference, it sits as **plaintext in GPU memory**. We built a bank vault with a screen door on the back.

The Geographic Trap

"Data sovereignty" today = a pin on a map. Your server sits in Vizag, but the **US CLOUD Act still reaches it**. Contractual promises, geographic boundaries, compliance checkboxes — comfort blankets with **legal trapdoors sewn into the lining**.

You Can't Legislate a Memory Dump

No NDA, no audit clause, no compliance framework prevents a hypervisor-level memory scrape. The gap is technical — and legal instruments cannot patch it.

The Three-Way Trust Dilemma

1

Model Owners

Fear weight theft and IP exfiltration

2

Infra Providers

Fear liability for customer data exposure

3

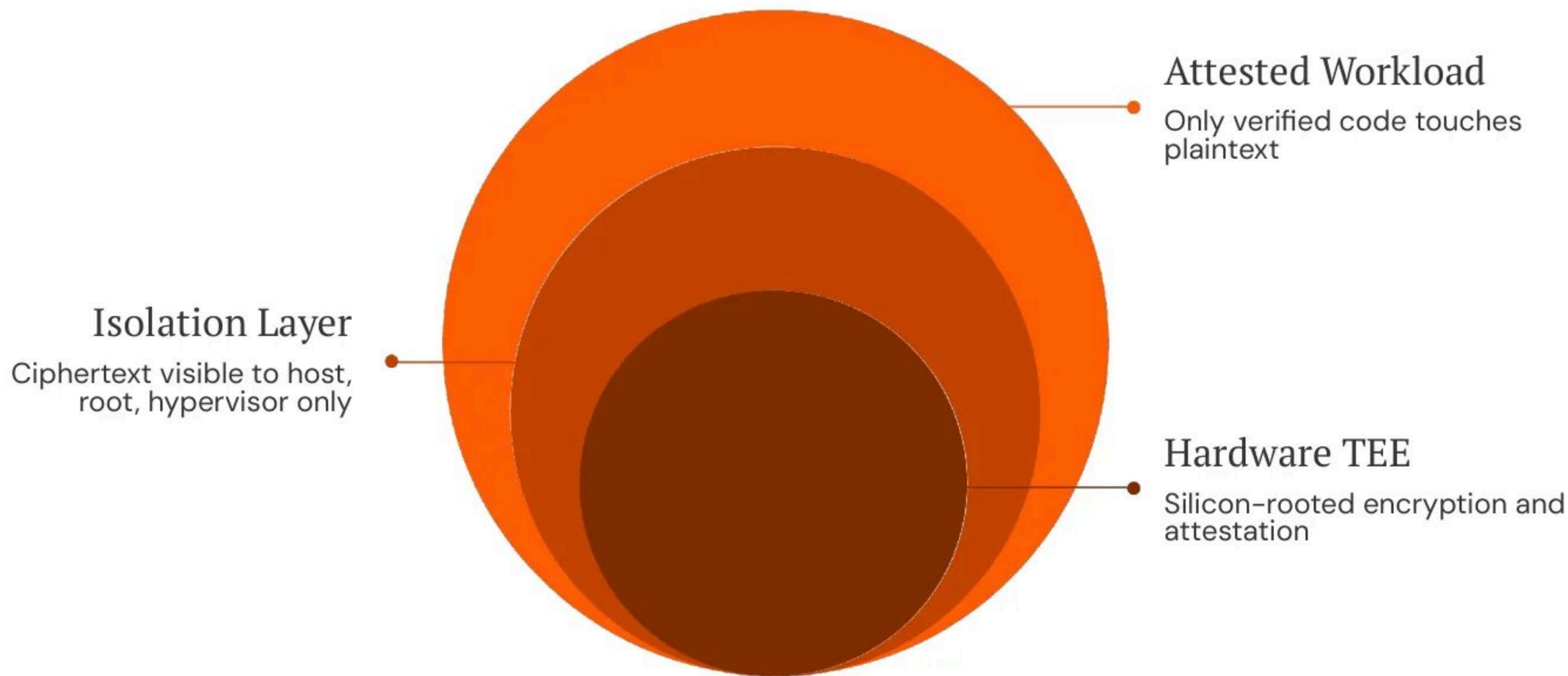
You

Fear your data in someone else's memory

Today's fix: NDAs and audit clauses — legal instruments patching a **technical gap**.

Confidential Computing: From "Trust Me" to "Verify Me"

Trusted Execution Environments (TEEs) are hardware-backed isolation zones that encrypt data even while it's being computed on. Root access, hypervisor, host OS — all see **ciphertext only**. Only the attested workload touches plaintext. This solves all three trust fears in one motion: weights protected, code provable, data sealed.



The open verification stack is production-ready today: **CNCF Confidential Containers** · **Kata Containers** · **IETF RATS** · **Keylime** — cryptographic attestation checkable by anyone, not just the vendor's security team.

Prove It, Don't Promise It

UPI didn't win because someone *promised* the banks it was secure* — it won because **the protocol was open and the rules were verifiable**. Private, verifiable AI inference is that same opportunity, one layer up the stack.

The Old Way

Tokens burned into oblivion. Blind faith in vendor promises. Sovereignty as a slogan on a slide. Compliance as a checkbox with legal trapdoors.

The New Way

Open silicon-rooted trust. Cryptographic proof instead of blind faith. One attestation receipt satisfies **DPDP, GDPR, EU AI Act and any new regulation simultaneously** — it's math, not a vendor's word.

📌 **Stop asking your AI vendors to promise you sovereignty. Start asking them to prove it.** At Digital Public Infrastructure scale, this is the difference between sovereignty as theater and sovereignty you can hand a regulator and say: *verify it yourself*.

SESSION

Scaling Inference at NxtGen

Abhishek Kumar Singh

NxtGen



NxtGen at Red Hat Summit Atlanta 2026

Summit sessions, labs and engineering discussions play a crucial role in establishing our engineering practices for the year for AI services.

KEY TAKEAWAYS

01 One replica efficiency

vLLM scheduling, batching and KV memory

02 Fleet intelligence

placement informed by cache, load and SLO state

03 Sovereign operations

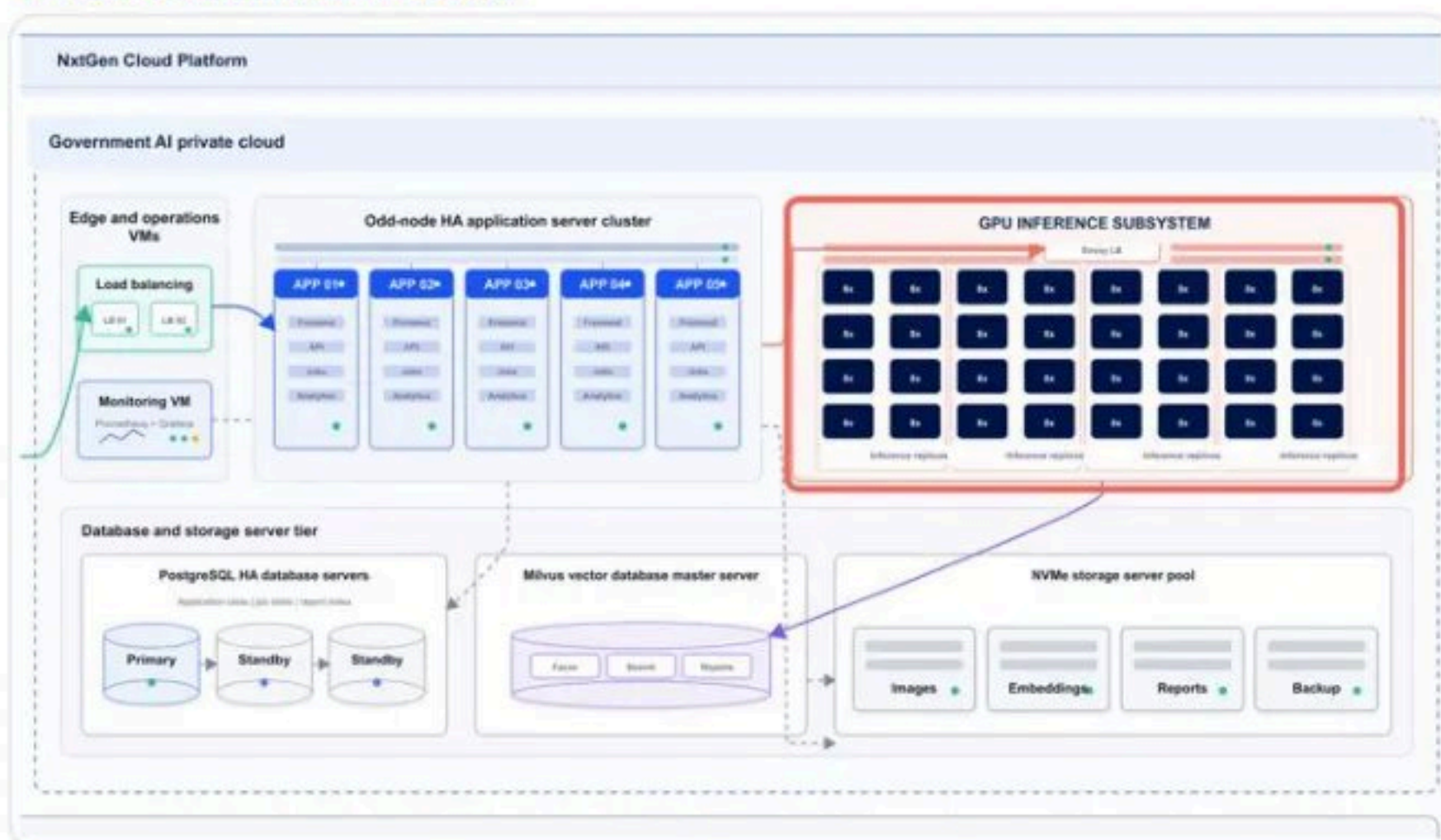
mixed accelerators managed as one measured service



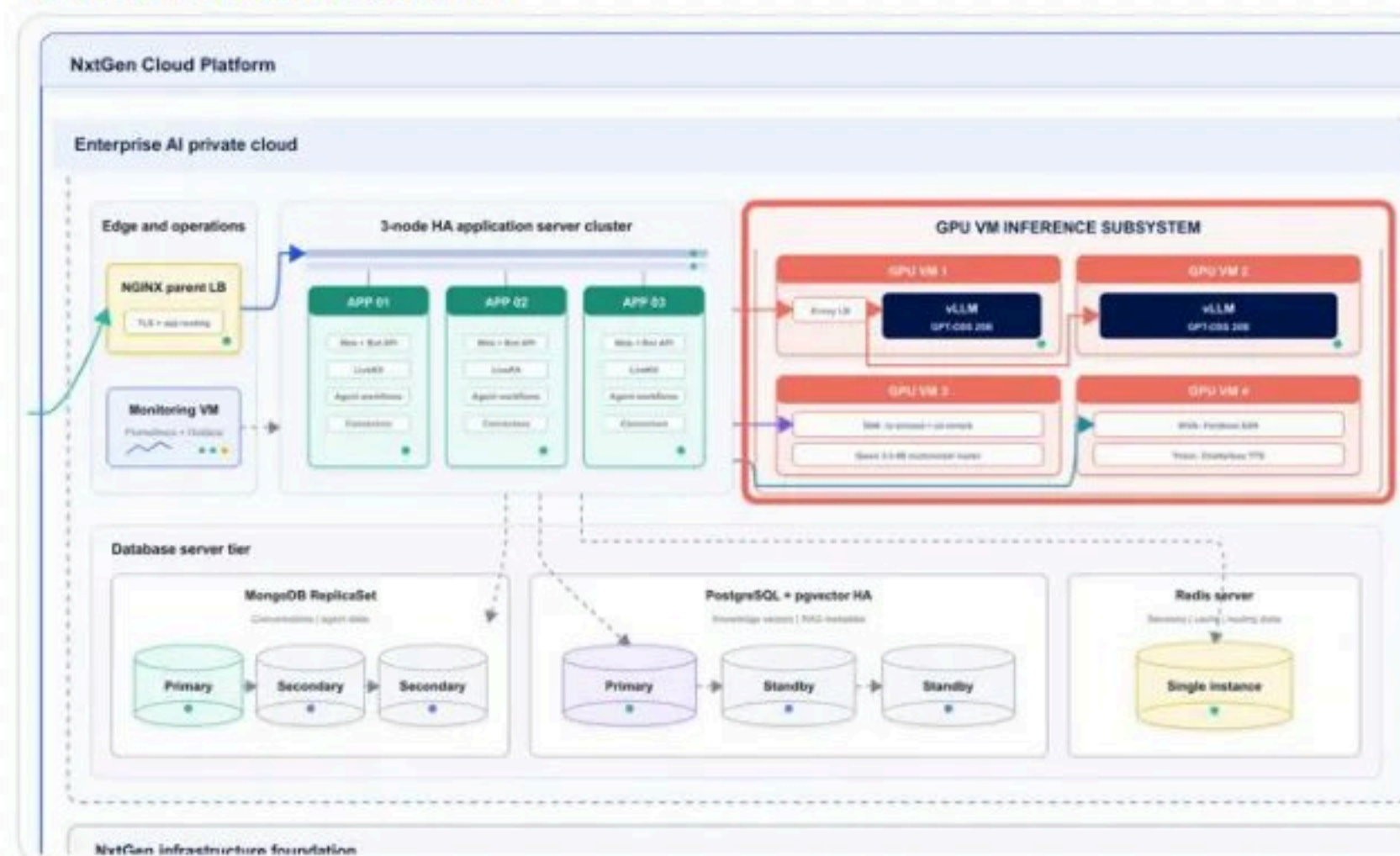
Inference inside the customer architecture

GPU serving is one subsystem in a much larger application and data platform.

TYPICAL GOVERNMENT CUSTOMER



TYPICAL ENTERPRISE CUSTOMER



Critically - Application, Database and GPU servers must scale out at the same rate.

vLLM makes one replica efficient

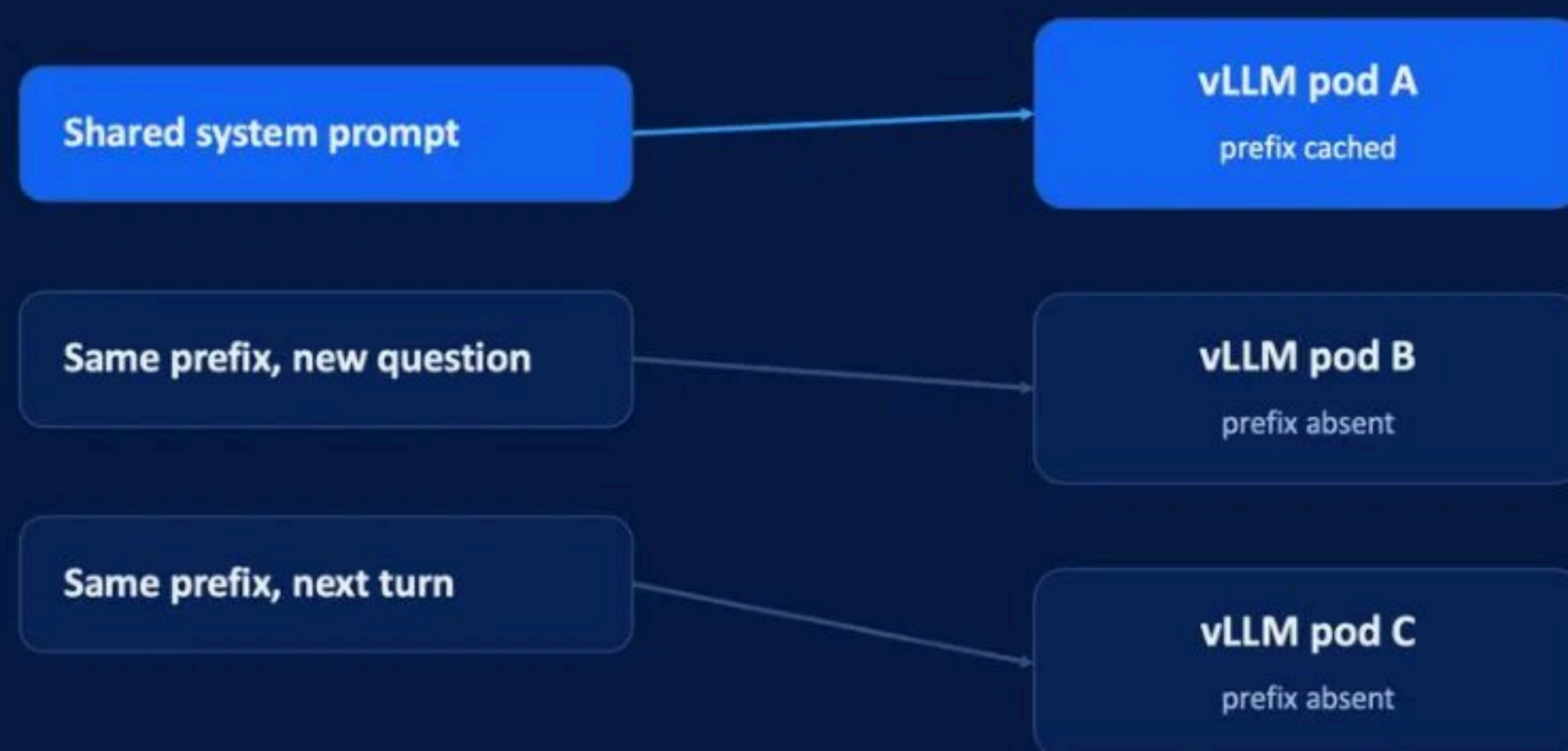
The engine removes waste inside a model server

- 01 **PagedAttention** Allocates KV memory in blocks so requests do not need contiguous slabs.
- 02 **Continuous batching** Admits and completes sequences dynamically as capacity changes.
- 03 **Chunked prefill** Interleaves long prompts with decode work to control interference.
- 04 **Automatic prefix caching** Reuses shared prompt blocks when the matching cache is local.

Optional levers: quantization for memory pressure, speculative decoding for decode latency, LoRA for model variants

Replica efficiency does not solve fleet placement

Round-robin preserves endpoint fairness, not KV locality or latency fairness
ROUND ROBIN



What repeats on the wire

Prefill work **computed 3 times**

Useful cache **stranded by pod**

Tail latency **set by cold turns**

At NxtGen in 2025, we used conventional load balancing across GPU VMs serving identical models. It answered requests but offered no continuity for multi-turn or long-horizon agents.

llm-d turns the fleet into an inference control plane

Filter, score and pick with the state that determines inference cost



Gateway API Inference Extension defines the pool. llm-d supplies the inference-aware decision.

Current ecosystem: llm-d v0.9, Gateway API Inference Extension v1.5

Model choice and replica choice are different decisions

A stable API can hide model, replica and hardware decisions without mixing their responsibilities



Earlier NxtGen solutions used rigid prompts to route agentic work to reasoning LLMs behind HA endpoints. Semantic Router and llm-d now separate model choice from replica placement.

Each scaling decision has a different success test

A single throughput number cannot validate model choice, replica placement and engine configuration

01

MODEL POLICY

Did the request reach the right capability and cost tier?

We could judge this via

Router precision and recall
Task quality by route

02

REPLICA POLICY

Did placement avoid cold cache and overloaded replicas?

We could judge this via

p95 TTFT and queue time
Cache hit and saturation

03

ENGINE + HARDWARE

Did the selected configuration improve useful capacity?

We could judge this via

Task quality and TPOT
Tokens per accelerator and cost

Benchmark the complete request path on NxtGen traffic before promotion.

NxtGen heterogeneous inference benchmark

NxtGen, IBM Research, Red Hat and evaluated two placement policies across three accelerator-vendor pools.

PREFIX-RICH REQUEST STREAM

The 7.2K-token prefix repeats while each question changes



1K output tokens per request

IIm-d EPP

Endpoint Picker

prefix-cache locality

replica saturation

Only the routing policy changed

ONE INFERENCE POOL

20 independent vLLM replicas, each with TP = 1

Vendor A

4 GPUs



Vendor B

8 GPUs



Vendor C

8 GPUs



The published benchmark anonymizes the hardware identities as Vendor A, B and C.

SAME PODS

both policies

SAME IMAGES AND FLAGS

within each pool

100G ROCE

shared fabric

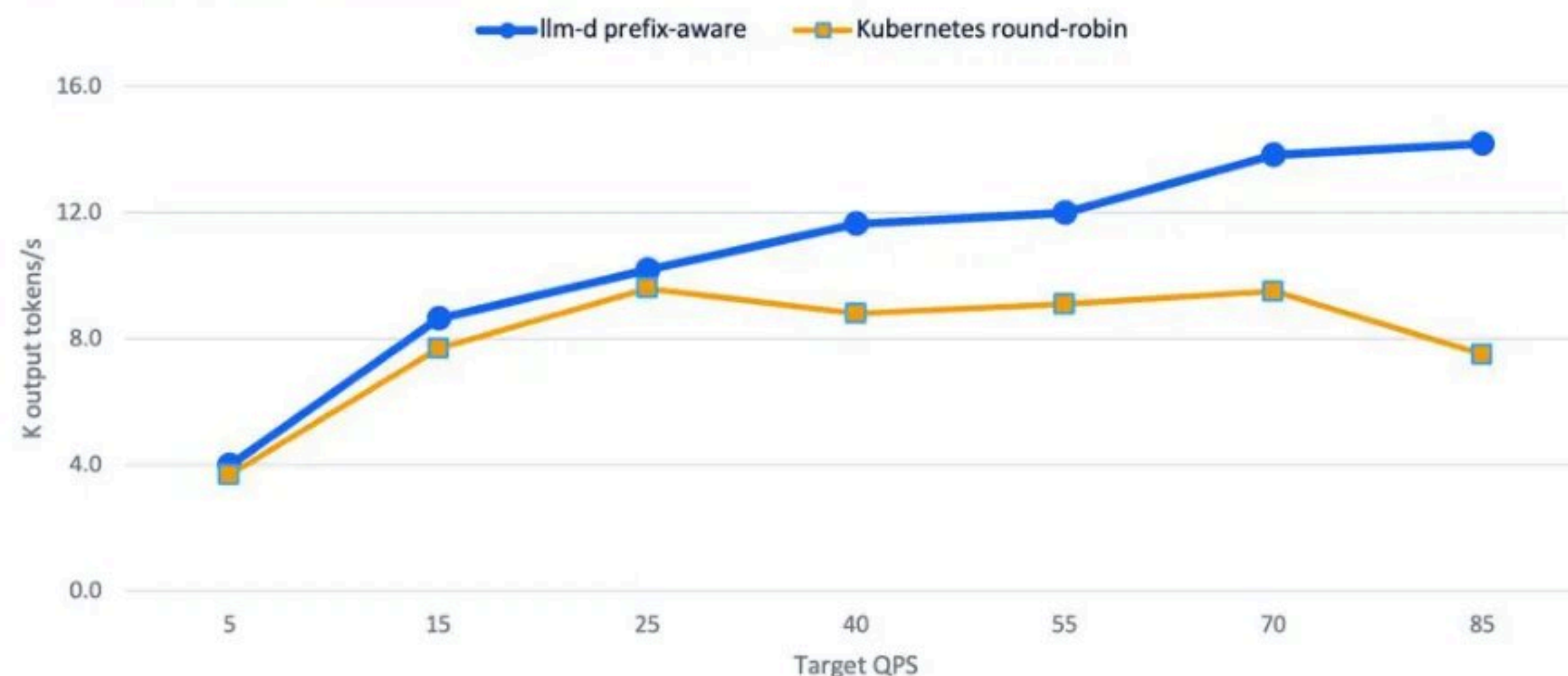
MODEL-SPECIFIC SCORER

precise or approximate

llm-d kept scaling after round-robin saturated

Granite 4.1 8B across 20 pods from three accelerator vendors, anonymized as Vendor A, B and C

Output throughput across the rate ladder



AT 85 QPS

14.2K
output tokens/s with llm-d

7.5K with round-robin

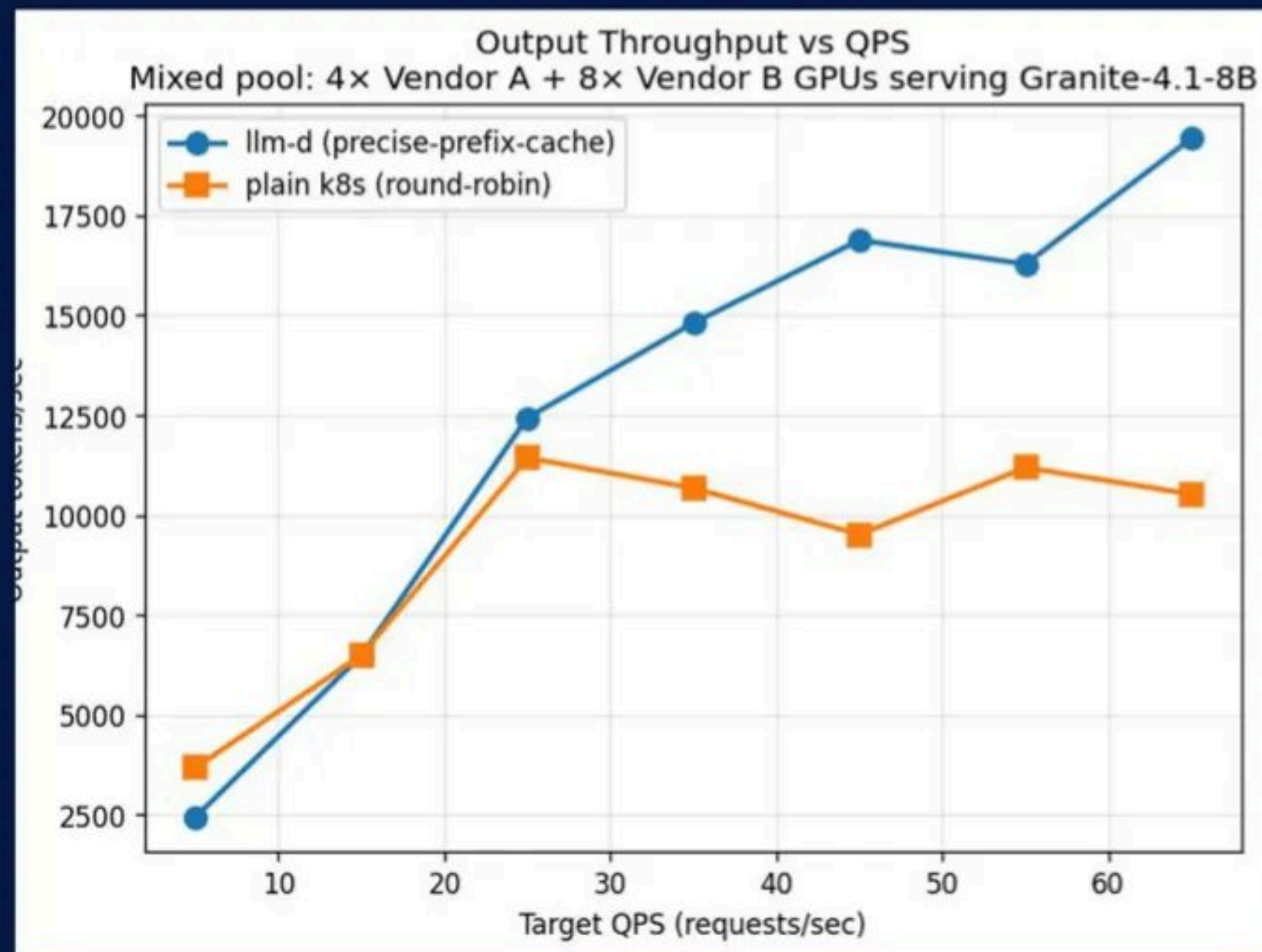
5.4x
faster TTFT
6.8s vs 36.4s

The EPP reached its CPU limit at 85 QPS before the serving pool reached its ceiling.

The routing advantage held across models and pool sizes

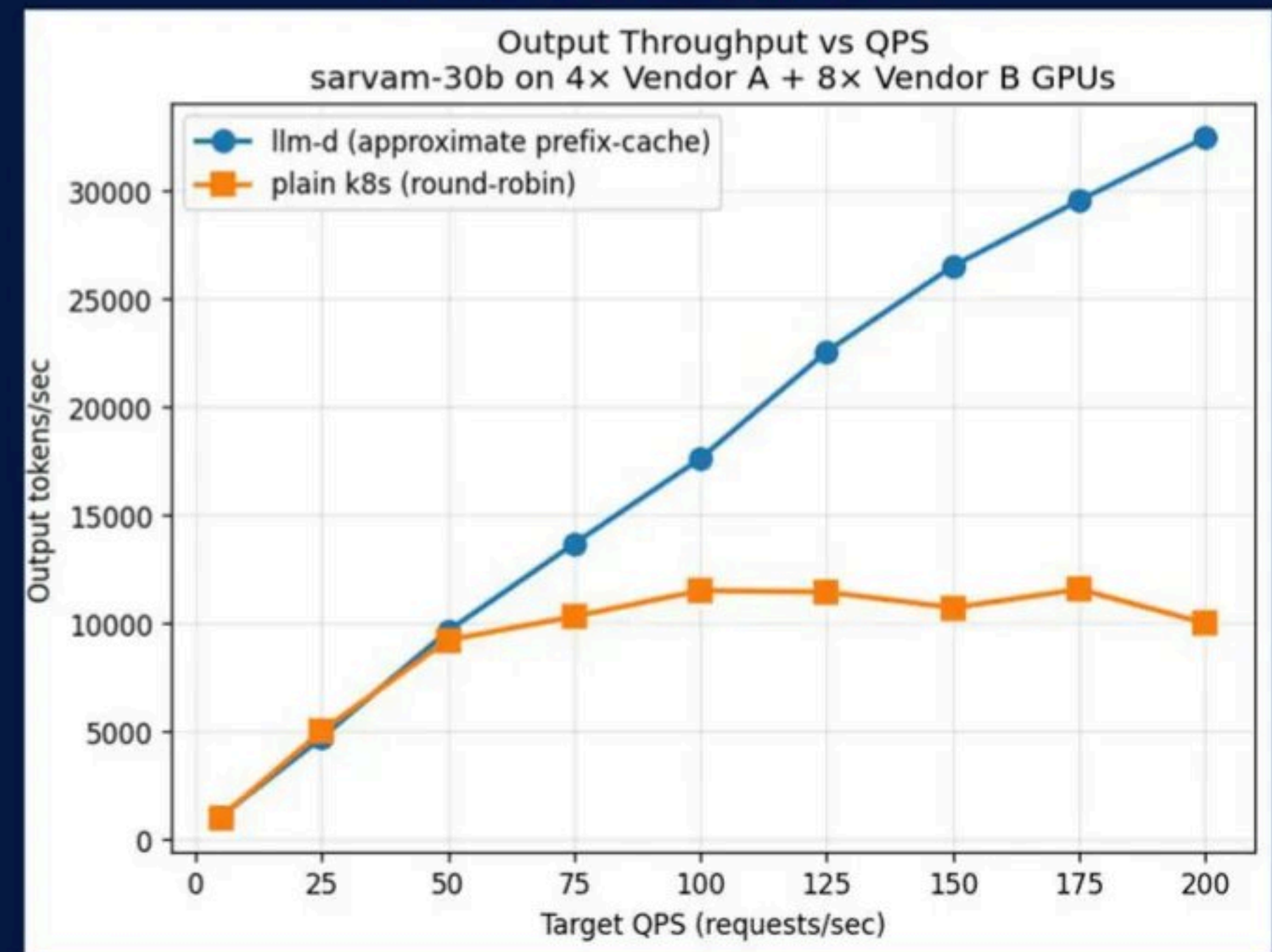
Two 12-pod mixed-vendor pools show the pattern with different models and prefix scorers.

GRANITE 4.1 8B, PRECISE SCORER



+85% throughput, 3.4–5.6x faster TTFT

SARVAM 30B MOE, APPROXIMATE SCORER



~3x throughput, 2.85–4.54x faster TTFT

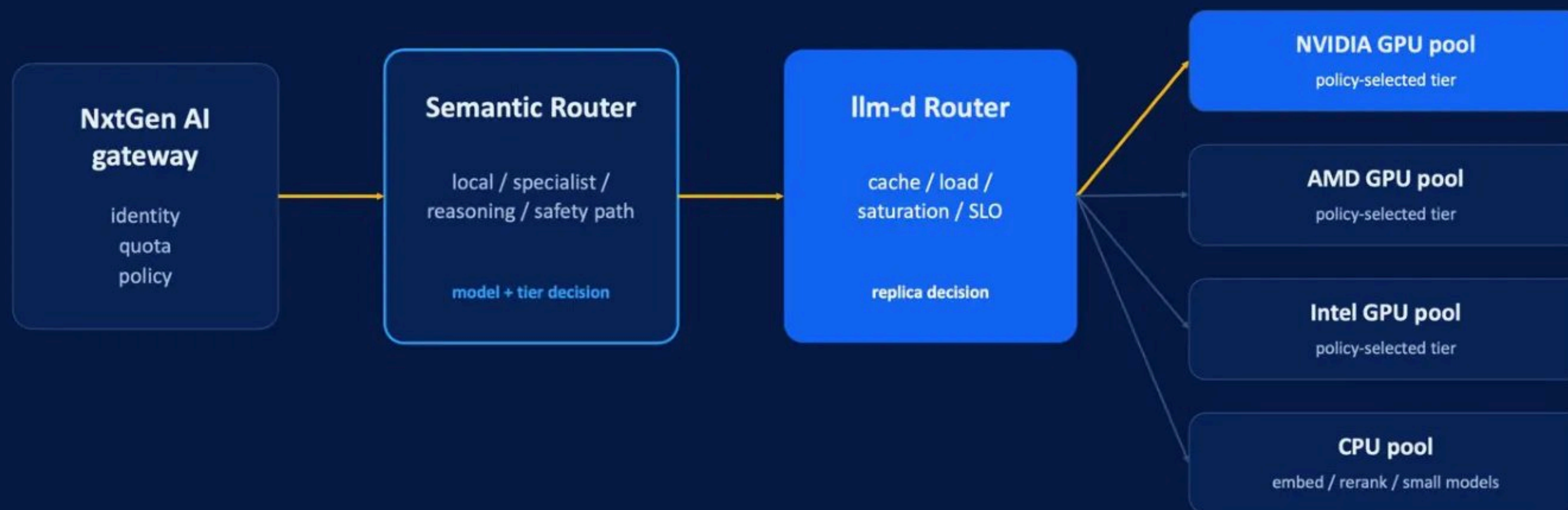
The ecosystem now covers the control-plane gaps

Current as of 16 September 2026



Ideal reference architecture

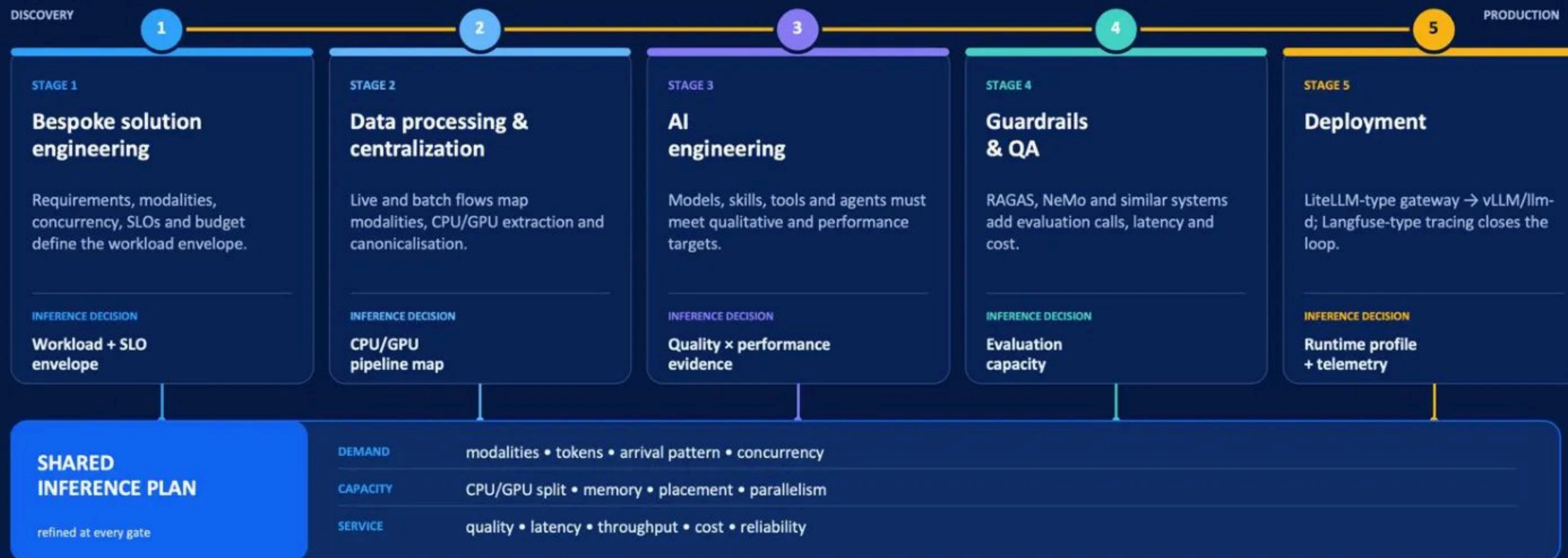
The benchmark validated replica placement. This proposal adds model and hardware-tier policy around it.



OpenShift supplies a common operating substrate across NVIDIA, AMD and Intel accelerators inside the sovereign boundary.

Every solutioning stage depends on the inference plan

NxtGen's patent pending Standardized AI Solutions Framework carries one measurable workload and service model from discovery to production.



At NxtGen - Inference is tagged to every tenant of AI Engineering – from the first conversation with the customer to deployment, and ahead.

SESSION

llm-d for Agentic Workloads

Distributed inference stack, optimization, and benchmarking for agentic workloads.

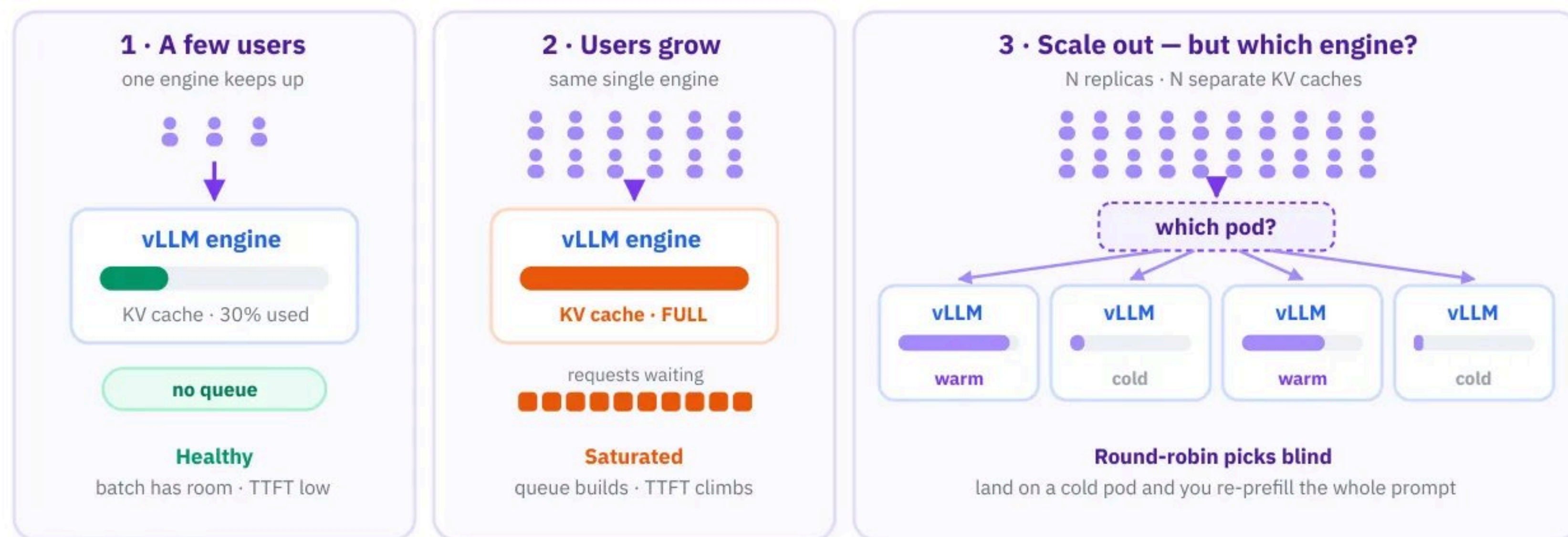
Pravein Govindan Kannan

IBM Research

Distributed LLM serving

1 • Load outgrows one engine

A single vLLM engine saturates — and replicas don't share a KV cache:



Scaling out is easy. Choosing **which replica** is not — round-robin sees neither cache nor load.

Distributed LLM serving

2 • Frontier models outgrow single GPU

Independent of load — wide EP and prefill/decode splitting are not optional at this size:



vLLM does each piece — **orchestrating them across nodes is a cluster problem.**

llm-d – Kubernetes, made LLM-aware

K8s-native distributed inference on **vLLM** + the Gateway-API Inference Extension – CNCF-track. **Started by IBM Research & Red Hat**, now built across the industry:

FOUNDING MEMBERS



Prefix-cache-aware routing

Prefill / decode disaggregation

SLO-aware routing

Workload autoscaling

Observability

Multi-vendor

TESLA

3× throughput

2× faster TTFT

aws

+70%

tokens / sec

ORACLE

+30%

throughput

Google

-40%

TTFT & ITL

Real production wins, across vendors.

The llm-d ecosystem

FOUNDING CONTRIBUTORS



LAUNCH PARTNERS

AMD

Cisco

Hugging Face

Intel

Lambda

Mistral AI

CONTRIBUTORS · UNIVERSITIES

DaoCloud

DigitalOcean

Moreh

Rebellions

UC Berkeley · Sky

UChicago · LMCache

USERS

Capital One

Tesla

Snowflake

Cohere

JetBrains

Prime Intellect

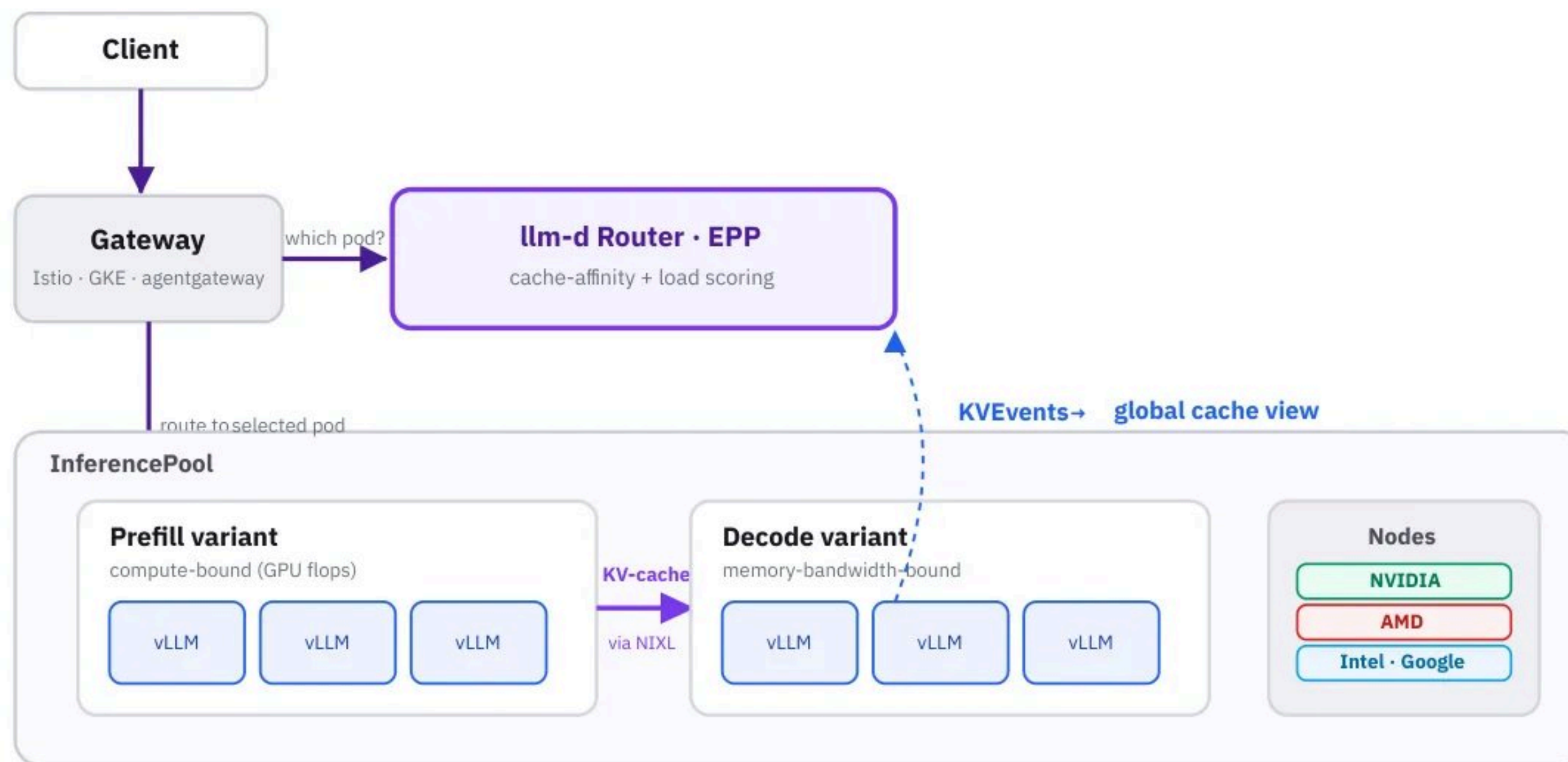
Reflection

Stelia

Novita AI

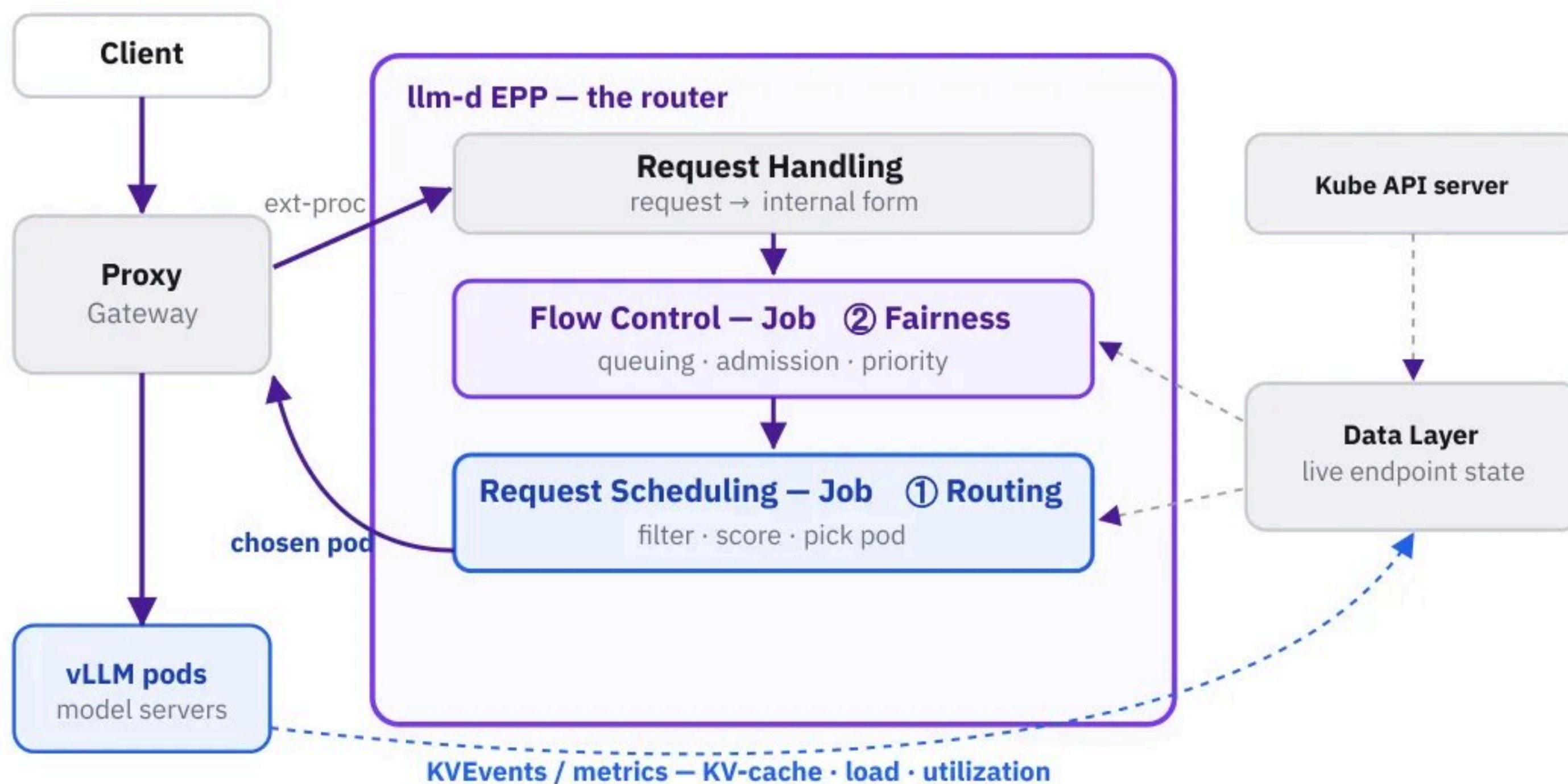
Open governance, CNCF-track.

llm-d architecture at a glance



The EPP does two jobs

- ① **Routing** — prefix- & load-aware placement · ② **Fairness** — queuing, admission & priority



How a pod actually gets picked

1 · PRODUCERS

Build the live view

KVEvents & metrics → who holds which prefix, how loaded each endpoint is

2 · FILTERS

Shrink the candidate set

hard constraints — role (prefill/decode), and shed endpoints that are too hot

3 · SCORERS

Rank what's left

soft preferences — weighted sum over cache hit, token load, active requests

Filters are veto. Scorers are preference. Highest total score wins the request.

All of it is **plugins** — you compose a policy.

The same thing, as configuration

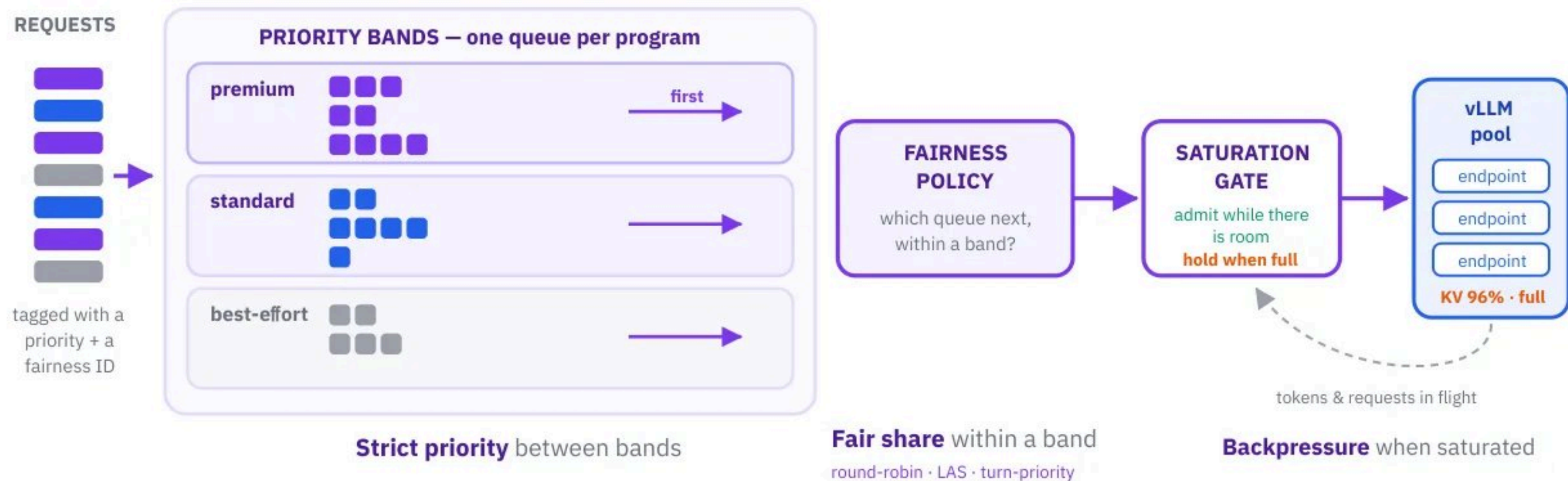
```
schedulingProfiles:
- name: prefill
  plugins:
  - pluginRef: prefill-filter          # FILTER · keep only prefill endpoints
  - pluginRef: prefix-cache-affinity-filter # FILTER · shed the hot ones
  - pluginRef: token-load-scorer        # SCORER · how many tokens in flight
  - pluginRef: cpu-prefix-cache-scorer  # SCORER · can it restore from DRAM?
- name: decode
  plugins:
  - pluginRef: decode-filter           # FILTER · keep only decode endpoints
  - pluginRef: active-request-scorer   # SCORER · who has a free slot
```

Prefill and decode get different policies

prefill is cache- and prefill-throughput-bound, so affinity earns its keep · decode is slot-bound, so what matters is which endpoint has room right now

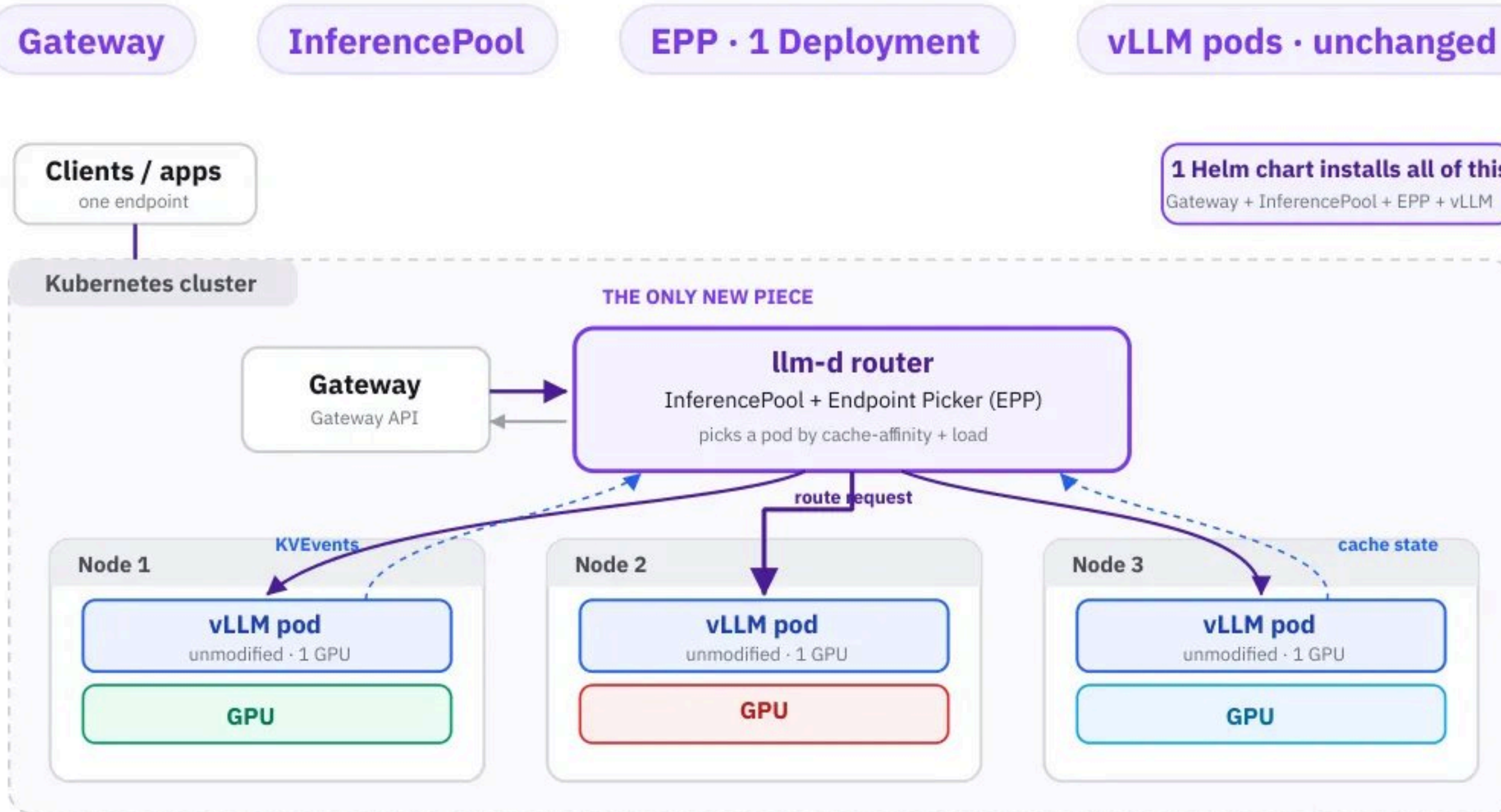
Flow control: admission, not just placement

When **every** endpoint is saturated, you decide **who waits**:



Routing decides where a request goes • flow control decides whether, and when

Deploying llm-d is just Kubernetes objects



Same pods, same vLLM, same Helm — **no app changes**. Only the routing layer is new.

Agentic workloads

GLM-5.2 on 64× H200 · flow control · **#sig-agentic-inference**

Agentic serving: prefill *is* the workload

GLM-5.2 · 744B MoE · 64× H200 · replaying **real Claude Code traces** (219 sessions, 78K reqs)

- ◆ input/turn p50/p90/p99 = **195K / 521K / 849K** tokens
- ◆ output p50 = **317** → **98.6% of tokens are input**
- ◆ **99.6%** median prefix reuse turn-to-turn
- ◆ **107** turns/session · sub-agents burst **7–51** at once

WHY THIS BREAKS NORMAL SERVING

Context grows past **750K tokens**, then **pauses** — 2 s p50, **26 s p90**. Live session state reaches **1.4–5.6×** GPU cache, so prefixes are evicted and every returning user pays a **full recompute**.

The unit of work is a **session**, not a request — and it is **prefill-bound**.

AT PRODUCTION SCALE

The same stack on 544× H100

GLM-5.2 · 544 H100s — not H200s · real agentic benchmarks · zero preemptions, zero serving errors

6.6M

peak output tok/min

3,000

concurrent agents

5–10×

cheaper than APIs

Local prefix cache hit rate

higher is better

approximate



44.5%

precise



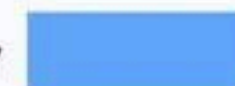
73.2%

Precise matching is worth +79% throughput and
–67% TTFT over approximate.

Prefill that must be recomputed

lower is better

GPU cache only



27.1%

+ CPU & P2P



14.8%

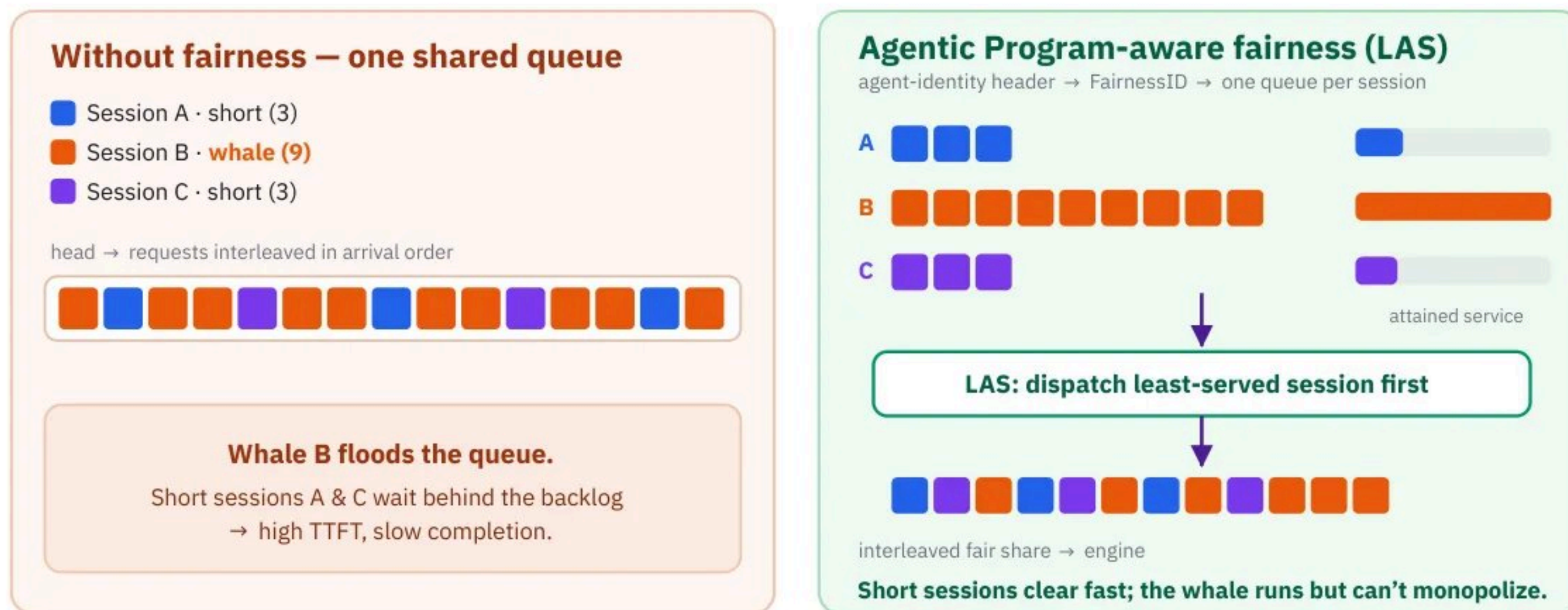
The extra tiers remove ~45% of the remaining
recompute — 85% of all input is served from cache.

Frontier-scale open models **don't need next-generation accelerators**

IBM Research · research.ibm.com/blog/running-open-models-on-h100-gpus-with-llm-d

Throughput isn't the whole story

Raising pool throughput doesn't decide **who** gets served:

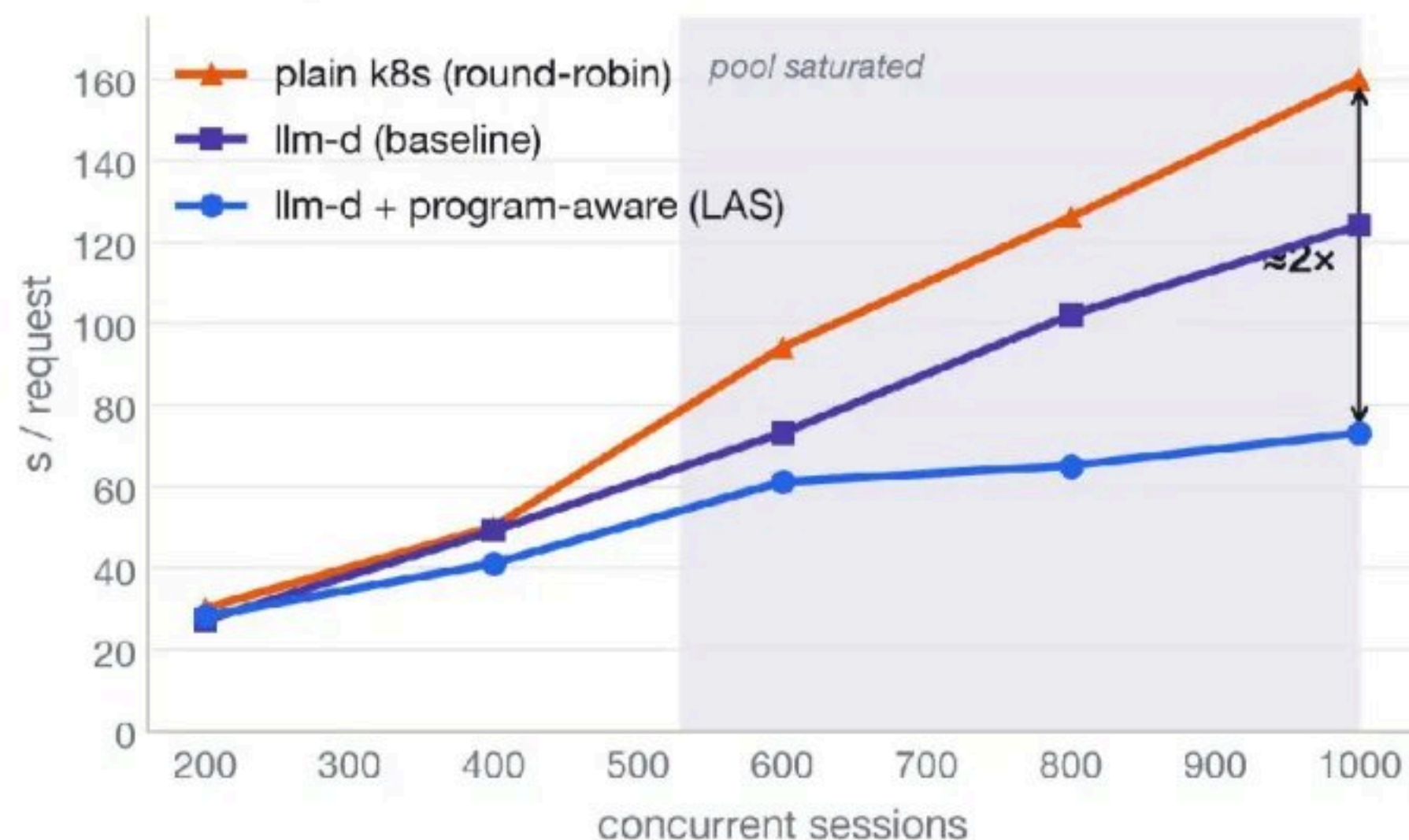


Standard header → **FairnessID** → one queue per session; **LAS** dispatches the least-served first.

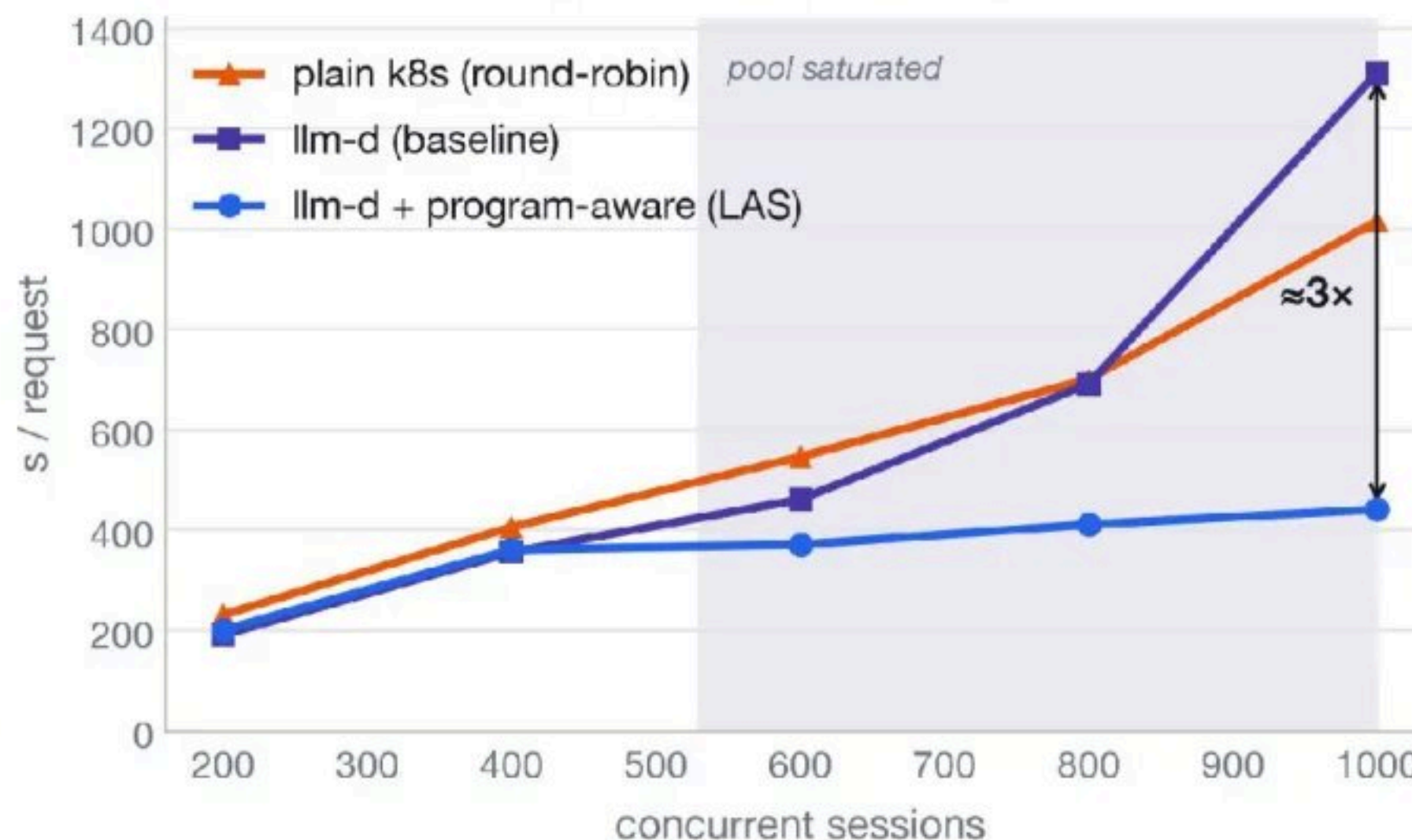
It works: fairer, at no throughput cost

Nemotron-120B · 16×H100 · real agentic traces (per-request completion = session_dur / requests)

Per-request completion — p50



Per-request completion — p99 (tail)



Turn-priority: schedule for prefix residency

Same per-program queues, one strategy switch — and the **opposite** objective:

score = turn number + (small weight × head wait) — dispatch the highest

LAS · EQUITY

Least-served session first

unit = weighted tokens — rescues short sessions stuck behind a whale

TURN-PRIORITY · EFFICIENCY

Deepest session first

unit = session depth — a deep session's prefix is still resident, so serving it re-prefills nothing

On the agentic replay: **~2× throughput** — repeated prefill comes back as capacity

Under contention it **sheds new sessions** rather than lose a deep prefix.

The agentic roadmap

- 1 **Session-aware routing** — keep a session on the pod that already holds its history
- 2 **KV-cache retention** — hold a session's cache across turns; evict by session state
- 3 **Session-graph scheduling** — schedule whole agent workflows, not single requests
- 4 **Agent hints** — let the agent signal intent to steer routing, caching & priority

The unit keeps growing: **requests → agents → sub-agents → multi-agents.**

Thank you — questions?

Get involved: llm-d.ai/community · github.com/llm-d

#SIG-Agentive-Inference is driving the agentive work — come build it with us.

llm-d.ai/blog

Pravein Govindan Kannan · pravein.govindan.kannan@ibm.com · github.com/praveingk

SESSION

Distributed Inference on ROCm

WideEP on vLLM & llm-d

Chaitanya Sri Krishna Lolla & Sirra Ajith

AMD



Meet the Speakers



Chaitanya Sri Krishna Lolla

Manager, Software Development — ML Frameworks Group, AMD

Leads the AI Model Performance and Distributed Inference India team, driving deep performance analysis and optimization across AI frameworks.



Sirra Ajith Prakash

Member of Technical Staff — AMD

Works on large language models, optimizing inference stacks such as vLLM on AMD Instinct MI GPUs, benchmarking agentic workloads, and profiling execution to identify bottlenecks.



Aditya Sivagnanam

Developer Relations, AIG Team — AMD

Over a decade of expertise working with customers as a Field Applications Engineer at AMD.

AMD AI Developer Program

Build Faster Learn Deeper Scale Smarter

AMD AI Developer Program Benefits

Cloud Credits

Start building with free cloud credits from **AMD Developer Cloud credits** or **Fireworks AI credits** when you join.

Community Access

Join our **Discord** and **forums** to get help, share ideas, and dive into technical conversations.

Learning Resources

- ✓ Explore **AI Academy** to access expert-led and self-paced AI courses.
- ✓ Enjoy one-month complimentary DeepLearning.AI pro membership.*

Showcase Opportunities

Share your projects for a chance to be featured on AMD Developer channels and recognized at AMD events

AI Developer Program provides the compute, expertise, and community developers need to accelerate AI innovation

*Terms and conditions apply see <https://www.amd.com/en/developer/ai-dev-program.html#faqs>.

AMD AI Developer Program

Who Should Join?

The AMD AI Developer program is built for developers who:

- Build or train AI models
- Optimize compute-intensive workloads
- Work with open-source frameworks
- Explore AI acceleration on AMD hardware



How to Join?

- Create your free AMD account and complete the AI Developer Program form
- Verify your email using the activation code and access the AMD AI Developer program portal
- Join the community channels to start connecting with fellow developers
- Submit the form to request your cloud credits to choose between AMD Developer Cloud credits or Fireworks AI Credits
- Stay tuned for member emails about events, training, and new resources

AMD AI Academy

Self-learning courses



AI Foundations

Build a strong foundation in artificial intelligence, machine learning, and generative AI concepts while learning how to run and experiment with AI models on AMD platforms.

8 Courses 15 Hour(s) 1586

[View Pathway →](#)



Build AI

Learn how to build intelligent AI-powered applications, agents, and multimodal workflows using modern frameworks, open-source models, and scalable AI architectures.

5 Courses 8 Hour(s) 377

[View Pathway →](#)



Deploy & Optimize AI

Deploy, scale, and optimize AI systems for production using AMD GPUs, modern inference frameworks, Kubernetes, and GPU performance optimization techniques.

6 Courses 7 Hour(s) 205

[View Pathway →](#)



Academic Developers

Learn how to empower teaching and research with AI through hands-on tutorials and modern techniques that enhance coursework, support students, and advance research.

7 Courses 14 Hour(s) 219

[View Pathway →](#)

<https://www.amd.com/en/developer/resources/training/amd-ai-academy.html>

AMD AI Academy example Course Curriculum

Introduction To AI Agents

2 Hours 12 Lessons 4 Projects 200 Points

Course Curriculum

Module 1: Introduction to AI Agents	1 Lessons
<> Video: Introduction To AI Agents SCORM Engine	
Module 2: Components and Architecture of AI Agents	1 Lessons
<> Video: Components And Architecture Of AI Agents	
Module 3: The Orchestration Layer	1 Lessons
<> Video: The Orchestration Layer	
Module 11: Hands On Lab 4	1 Lessons
Lab: Router Agent	
Module 12: Quiz	0 Lessons

Unenroll

Duration	2 Hours
lessons	12 lessons
Projects	4 hands-on
Level	Beginner
Certificate	Yes

AMD ROCm Certification Program

ROCm Certified Programs

The ROCm Certified Associate Digital Badge and Certificate will be awarded by email via Credly upon successful completion of the certification.



Available Now

ROCm™ Certified Associate

Core AMD ROCm™ Software Application Development (Associate)

Learn how to design software application code built around hands-on workshops focused on the design, optimization, and deployment of AI on AMD Instinct™ GPUs.



Coming soon

ROCm™ Certified Professional

Advanced AMD ROCm™ Software Development and Production Readiness (Professional)

Gain certification validating your ability to design, optimize, and deploy production-scale distributed AI training and high-performance inference workloads across multi-GPU and multi-node environments.



Coming soon

ROCm™ Certified Expert

Architecture-aware AMD ROCm™ Software Performance Optimization and Scale (Expert)

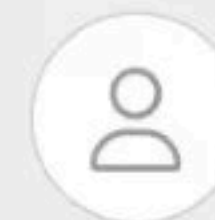
Learn how to design, deploy, and operate production AMD GPU infrastructure, from scaling AI workloads and GPU pipelines to managing and automating GPU clusters in real-world environments.

Certification Overview

Core AMD ROCm™ Software Application Development - Certified Associate



Search by badge name, org



This badge was issued to [Aditya Sivagn](#)

Date issued: September 18, 2026 | Exp



Earn this badge

Start your AI journey with AMD Developer Playbooks™

AMD Playbooks Playbooks Models Docs ? Login

Start your AI journey with AMD Developer Playbooks™

Step-by-step playbooks and powerful AI models ready to run. Choose your device to get started.

AMD Ryzen™ AI Halo Ryzen™ AI APUs Radeon™ GPUs All

FEATURED beginner

Generating images with ComfyUI and Z Image Turbo

Create stunning AI-generated images using ComfyUI with Z Image Turbo on STX Halo™

15 MIN Start Learning →

Automating Workflows with n8n and Local LLMs
Build an AI-powered news summarizer using n8n and Lemonade on STX Halo™
30 MIN beginner

Local LLM Coding with VSCode and Qwen3-Coder
Use VSCode with locally-running Qwen3-Coder for private code assistance on STX Halo™
1 HR intermediate

Running and serving LLMs with LM Studio
Set up LM Studio and LM Studio Server to run and serve large language models on STX Halo™
30 MIN beginner

Running LLMs with PyTorch and ROCm
Learn to run powerful language models on your PC with PyTorch and ROCm to summarize documents quickly and easily
30 MIN beginner

Building Your First Agent with GAIA
Build a 100% local AI agent — no cloud APIs needed. Use the GAIA SDK to create a hardware advisor on your STX Halo™
20 MIN intermediate

Clustering Two STX Halos with llama.cpp RPC
Set up distributed inference using RPC server across two STX Halo™ devices with llama.cpp to run 350B+ models
1h 30m advanced

Show All Playbooks (23)

[AMD Playbooks -](http://playbooks-dev.amd.com/#playbooks)

<http://playbooks-dev.amd.com/#playbooks>

AMD University Program (AUP) AI & HPC Cluster for Researchers

Slurm cluster comprised of AMD EPYC CPUs and AMD Instinct GPUs (MI210, MI250, MI300X, MI325X, MI350X) connected with a GPU-aware interconnect.

- ROCm for running AI & HPC workloads on Instinct
 - Heterogeneous-Computing Interface for Portability (HIP)
 - Familiar CUDA-like syntax (e.g., hipMalloc, hipMemcpy) to develop portable source code for AMD and NVIDIA accelerators
 - hipify tools for automatic translation from CUDA to HIP
 - Profiling/debugging tools, GPU-accelerated libraries, etc.
- Parallel file system for IO and persistent storage
- ROCm-enabled PyTorch
 - Runs out of the box on ROCm with Day 0 support

Node-hour allocations available to academic researchers and educators

- PIs must be academic faculty or postdocs
- Application-level work (e.g., no `sudo`)



<https://www.amd.com/en/corporate/university-program/ai-hpc-cluster.html>

AMD **EPYC** **INSTINCT** + **ROCm**



Developers: AMD Developer Cloud

Pay-as-you-go platform to provide easy access to 1x- or 8x-MI300X GPU VMs for developers

- Compute credits available to get started
- Docker containers (pre-installed or custom) available to simplify setup
- Snapshots to save progress and restart where you left off



Two AMD Instinct MI300X GPU Instance Configuration

Small Configuration: 1 x GPU

- GPU Memory: 192 GB
- vCPUs: 20 vCPUs
- System Memory: 240 GB
- Boot Disk: 720 GB NVMe
- Scratch Disk: 5TB- NVMe

Large Configuration: 8 x GPU

- GPU Memory: 1536 GB
- vCPUs: 160 vCPUs
- System Memory: 1920 GB
- Boot Disk: 1920 GB NVMe
- Scratch Disk: 40 TB- NVMe

Create an Account





Distributed Inference on ROCm with WideEP on vLLM & Ilm-d

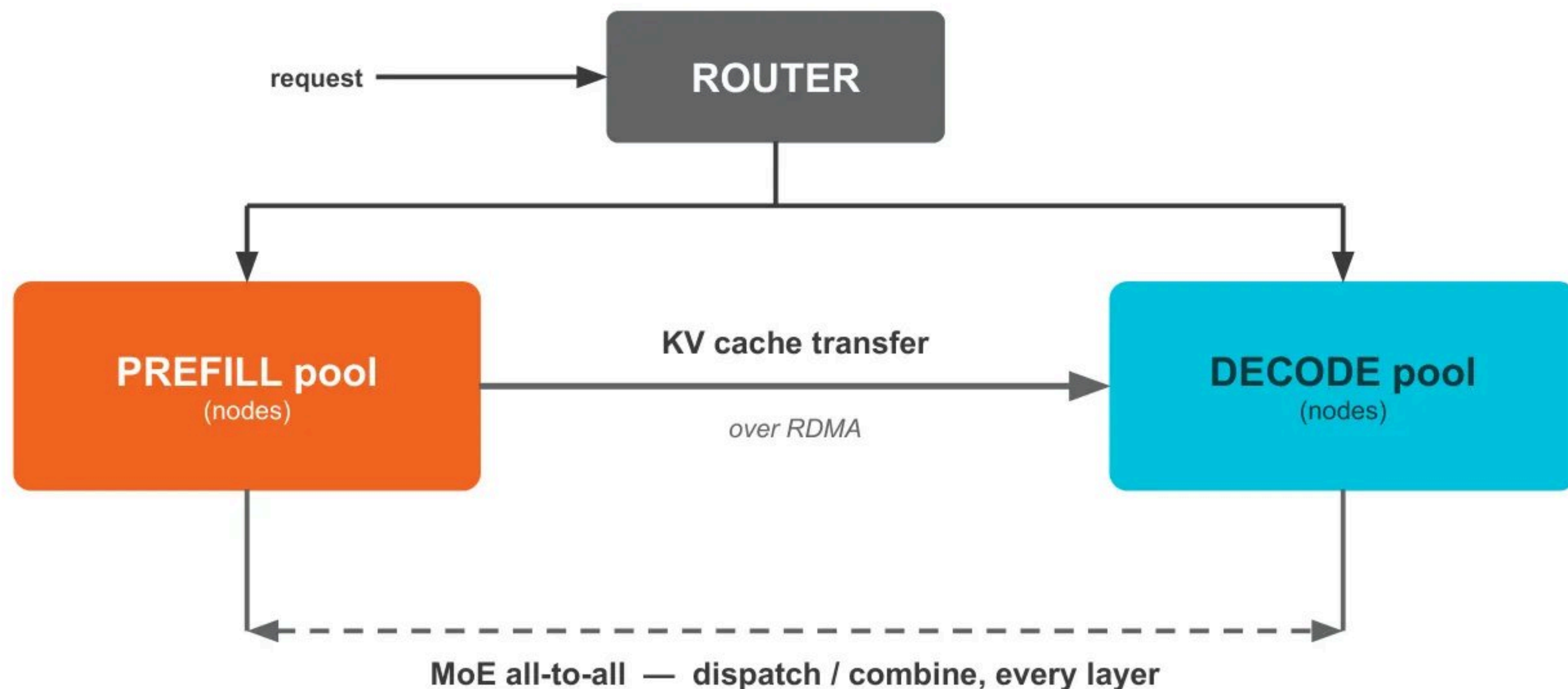
Chaitanya Sri Krishna Lolla
Manager Software Development

AMD 
together we advance_

Acknowledgements

*Mathews Aswin, Sun Peng, Li Chuan, Ainapurapu Satya Ramji,
Ravi Gupta, Shiksha Patel, Roche Kenny, Cave Vincent, Douglas Lehr,
Ai Fangzhou, Li Li, Akbarzadeh Amir, Tej Kiran, Avinash Paul, Nitin Singh, Mir Ali,
Barakat Basem, Prateek Kumar*

What is distributed inference?



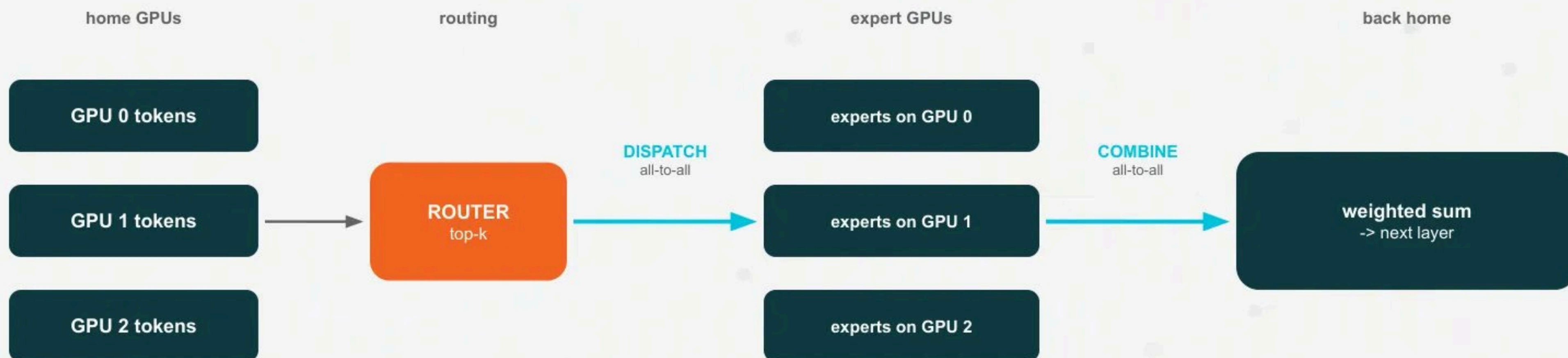
- One model, many GPUs and many nodes — it no longer fits, or keeps up, on one
- Prefill and decode want opposite things — split the pools, move the KV cache
- Experts spread across the cluster — every layer pays an all-to-all

Two pools, one KV hop, an all-to-all in every MoE layer.

The shape of MoE communication

- Dispatch: token to its expert's GPU. combine: partial result back to the home GPU
- Internode normal kernels for prefill throughput; low-latency IBGDA kernels for decode

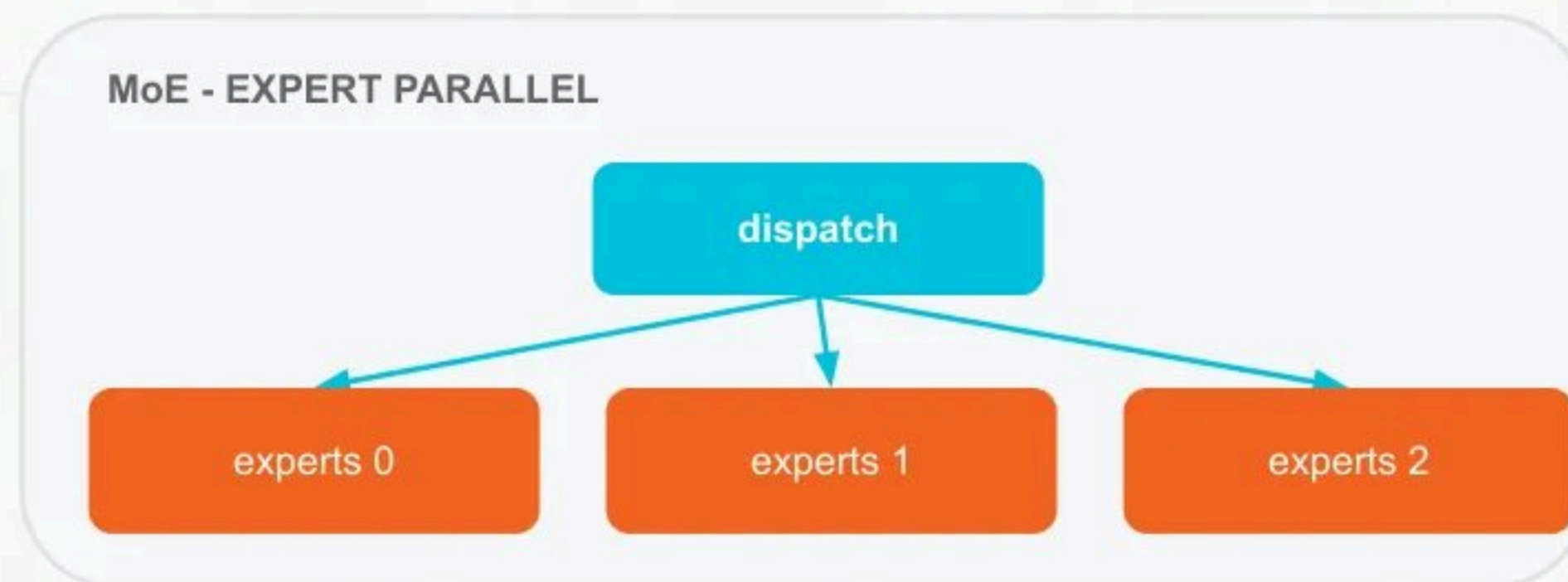
Two all-to-alls per MoE layer, per token, on the critical path.



DeepEP and MoRI-EP fuse routing, XGMI and RDMA into exactly these two kernels

Wide EP: DP attention + EP MoE

- **Attention runs data-parallel** — each rank owns its requests and their KV. No cross-rank attention traffic, no KV duplication.
- **MoE runs expert-parallel** — experts spread over *all* ranks
- One communication pattern instead of two competing ones
- vLLM “*--data-parallel-size*” + “*--enable-expert-parallel*”
- All-to-all backend on ROCm: “*--moe-a2a-backend mori_high_throughput | mori_low_latency*”



Why “wide”, and what it costs

- **Wide** = the expert-parallel group spans **many nodes**
- This is the direct answer to “why multi-node?”
- More GPUs → fewer experts each → more HBM free for KV cache and batch size
- **The cost:** the all-to-all now crosses the network, and its participant count grows
- MoRI-EP dispatch: 307 GB/s XGMI at EP8; at EP16, 171 GB/s XGMI + 52 GB/s RDMA [1]

WIDER EP - WHAT MOVES



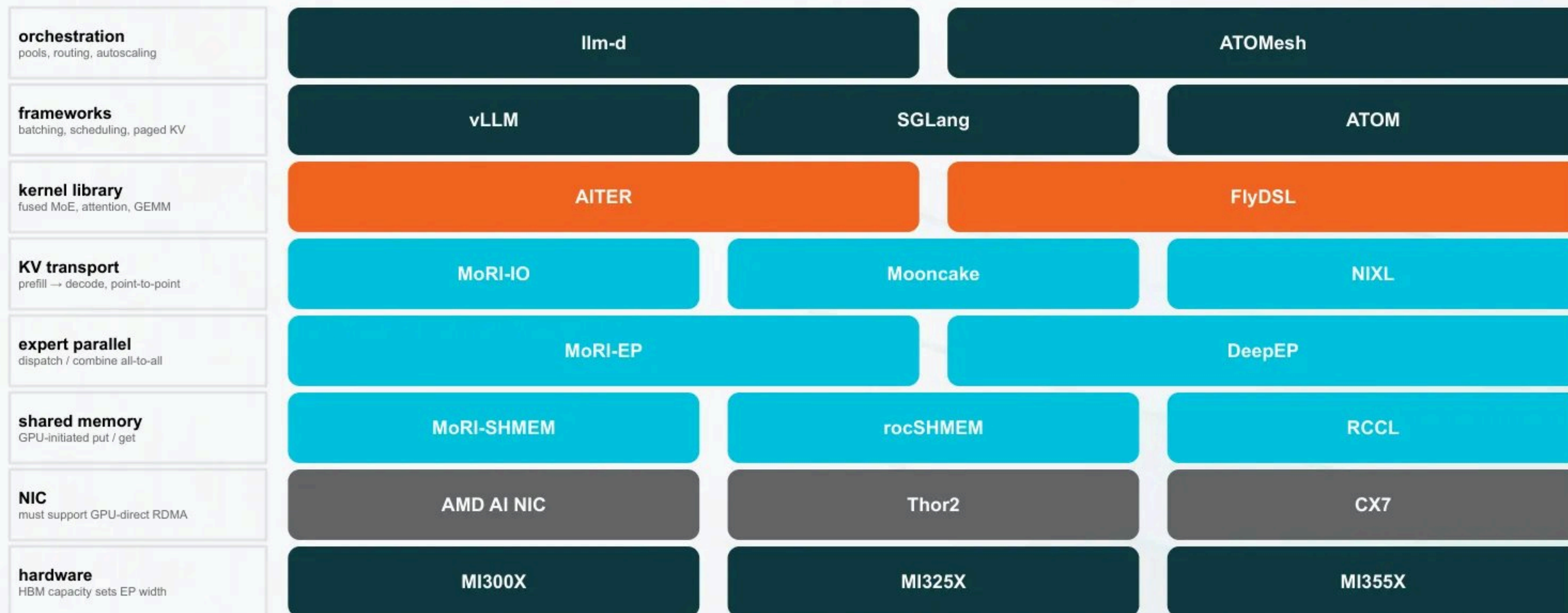
[1] MoRI-EP dispatch benchmarks — github.com/ROCm/mori#benchmarks

Configuration: MoRI-EP dispatch/combine microbenchmark, 8× AMD Instinct™ MI300X GPU per node with ConnectX-7 NICs; EP8 = 1 node, EP16 = 2 nodes; 4096 tokens, 7168 hidden, top-8 experts, FP8 dispatch / BF16 combine, DeepSeek-V3 configuration. Results may vary.

The ROCm distributed inference stack

ROCm Distributed Inference Framework Software Stack

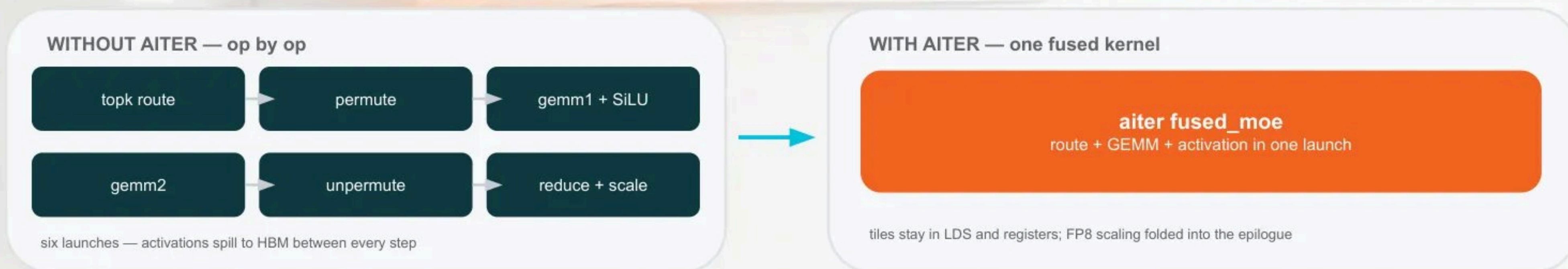
Three communication bands, not one — KV transport, expert parallel and shared memory get conflated constantly; the rest of this section takes them one at a time, bottom up



same stack whether you run vLLM, SGLang or ATOM — only the top two bands change

AITER

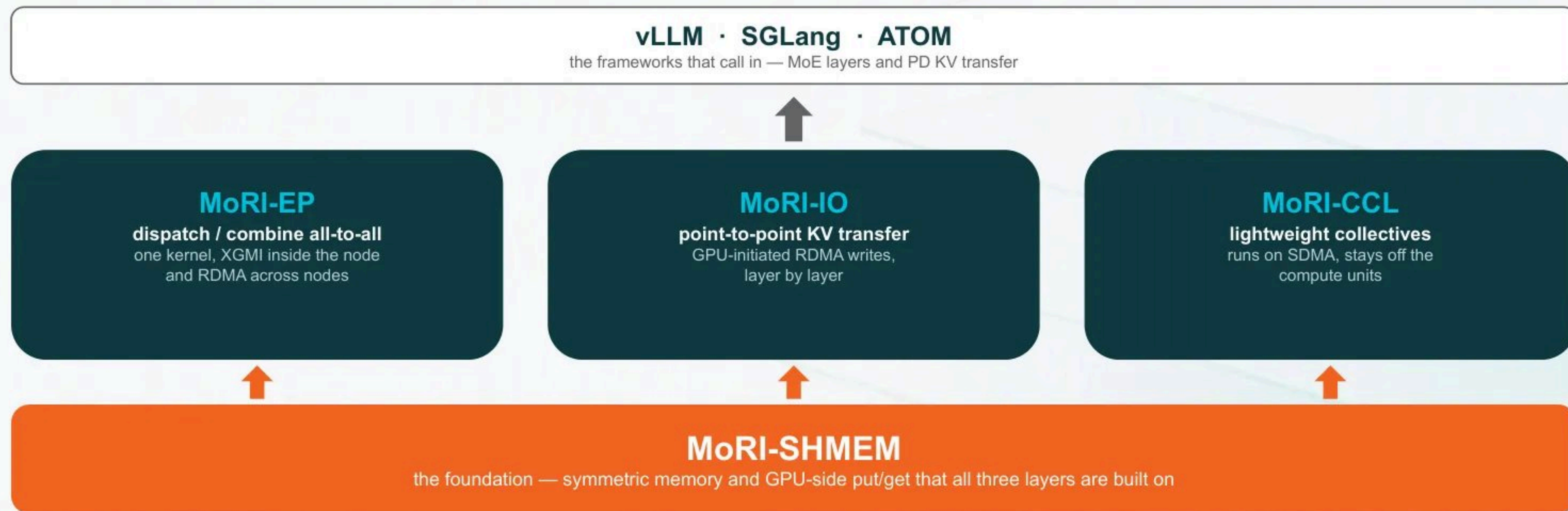
- AMD's ROCm kernel library – fused MoE, attention, GEMM, RMSNorm and quantization ops
- Four kernel tiers – HIP C++, Triton, FlyDSL and hand-written assembly – behind one Python/C++ op API
- Also loads under vLLM and SGLang as a ROCm kernel backend, so engine code stays unchanged
- Fused kernels remove the HBM round trips between routing, permute, GEMM, activation and scaling
- Per-arch tuned configs for MI300X/MI355X, selected at runtime by shape, dtype and expert count
- Block-scaled FP8 and INT4 paths with scaling folded into the GEMM epilogue, plus paged-attention and MLA kernels



same story for attention: QKV, RoPE, paged KV read and softmax collapse into one AITER kernel

MoRI — a layered stack, not a monolith

ONE STACK, FOUR LAYERS — EACH ONE USABLE ON ITS OWN



XGMI inside the node · RDMA across nodes · SDMA for the quiet path

take the whole stack, or take one layer — MoRI-IO alone is a valid KV connector

MoRI-EP + AITER fused MoE — one layer end to end

- One MoE layer is three things: dispatch all-to-all, expert GEMM, combine all-to-all
- MoRI-EP owns both all-to-alls — one kernel, XGMI in-node and RDMA across
- AITER fuses the expert side — no HBM round trips between routing, GEMM and scaling
- Split send/recv overlaps dispatch for micro-batch N+1 with the GEMM for N; kernel choice is the knob — InterNodeV1 for prefill, V1LL/AsyncLL for decode



comm and compute run on separate queues — MoRI-EP dispatch/combine overlaps the GEMM

Recent upstream work

Lessons from scaling GLM-5.1-FP8 to 64 GPUs

- Full prefill/decode disaggregated Wide-EP serving for GLM-5.1-FP8 on MI300X, over AMD MoRI
- Model shape: 78 layers, 256 experts top-8, MLA plus DeepSeek Sparse Attention
- Scaled to 64 MI300X GPUs — EP8 through EP32, all-to-all on both the prefill and decode side [1]
- Two upstream enablement items landed first: per-request attention metadata, and full indexer KV transfer
- Hardware: MI300X (gfx942), 8 GPUs per node, RoCEv2 fabric

WHAT IT TOOK — FOUR PIECES

Model Enablement

MLA and DSA attention running under Wide-EP disaggregation, not just single-node

Attention Metadata

vLLM #47766 keys metadata per request, keeping the persistent sparse-MLA kernel on the fast path

Indexer KV transfer

the connector transfers all 156 caches, 78 main plus 78 indexer

Transport

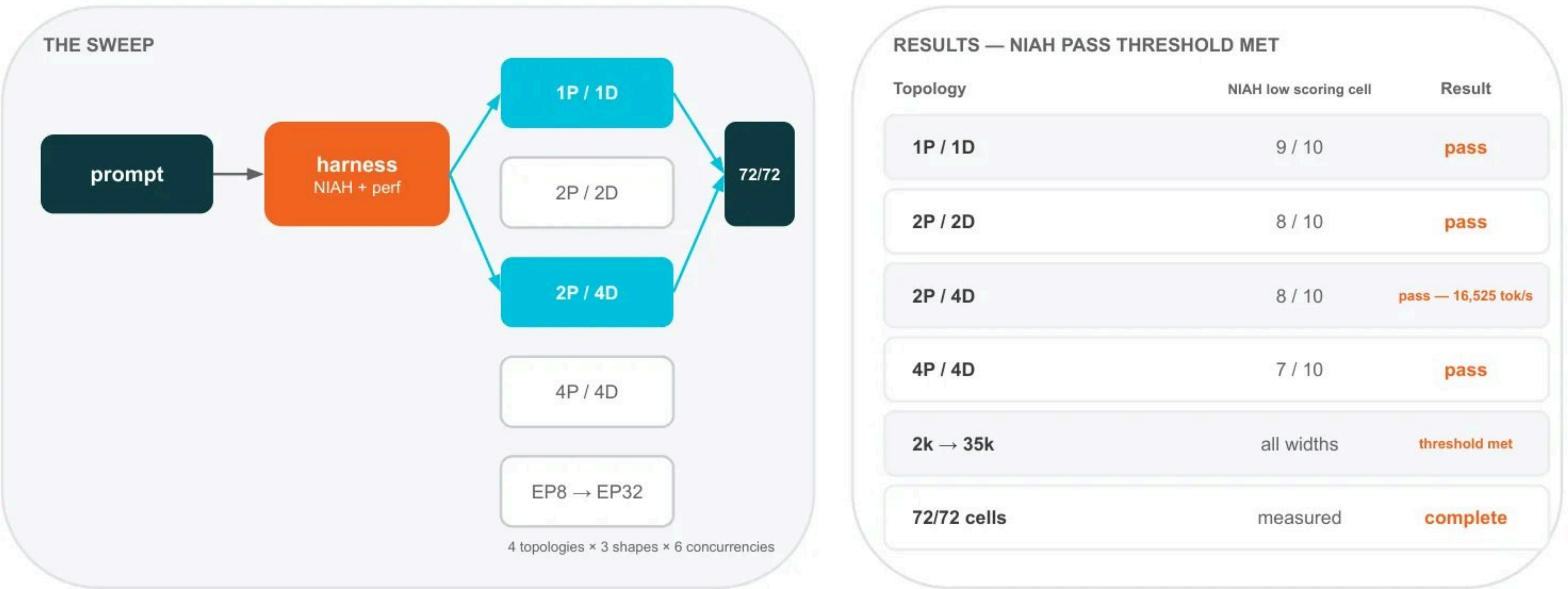
MoRI-EP for the two all-to-alls, MoRI-IO for the RDMA KV hop between pools

the enablement is upstream, not a private branch — this runs on the stock vLLM ROCm image plus MoRI

[1] ROCm blog, "DI Series: Scaling GLM-5.1-FP8 to 64 MI300X GPUs" — rocm.blogs.amd.com/software-tools-optimization/di-glm-wideep

GLM-5.1-FP8 — the complete 72/72 benchmark matrix

Four topologies, three input shapes, six concurrencies. All 72 cells measured; all four topologies met the NIAH pass threshold from 2k to 35k. [1]



The 7–9/10 dips are single-needle variance at a fixed seed. Retrieval met the NIAH pass threshold across the full 2k–35k range at each EP width tested, on both the prefill and decode all-to-all.

[1] ROCm blog, “DI Series: Scaling GLM-5.1-FP8 to 64 MI300X GPUs” — rocm.blogs.amd.com/software-tools-optimization/di-glm-wideep
Configuration: 8× AMD Instinct™ MI300X GPU (gfx942) per node, RoCEv2 fabric, eight GPU-local backend NICs; vLLM built from source (wide-EP WRITE mode + PR #47766), AITER e03fa6040, MoRI 42e895472b08; GLM-5.1-FP8. Benchmark results are preliminary; actual performance may vary based on configuration, software versions, drivers, and optimizations.

KV Capacity sets the concurrency ceiling

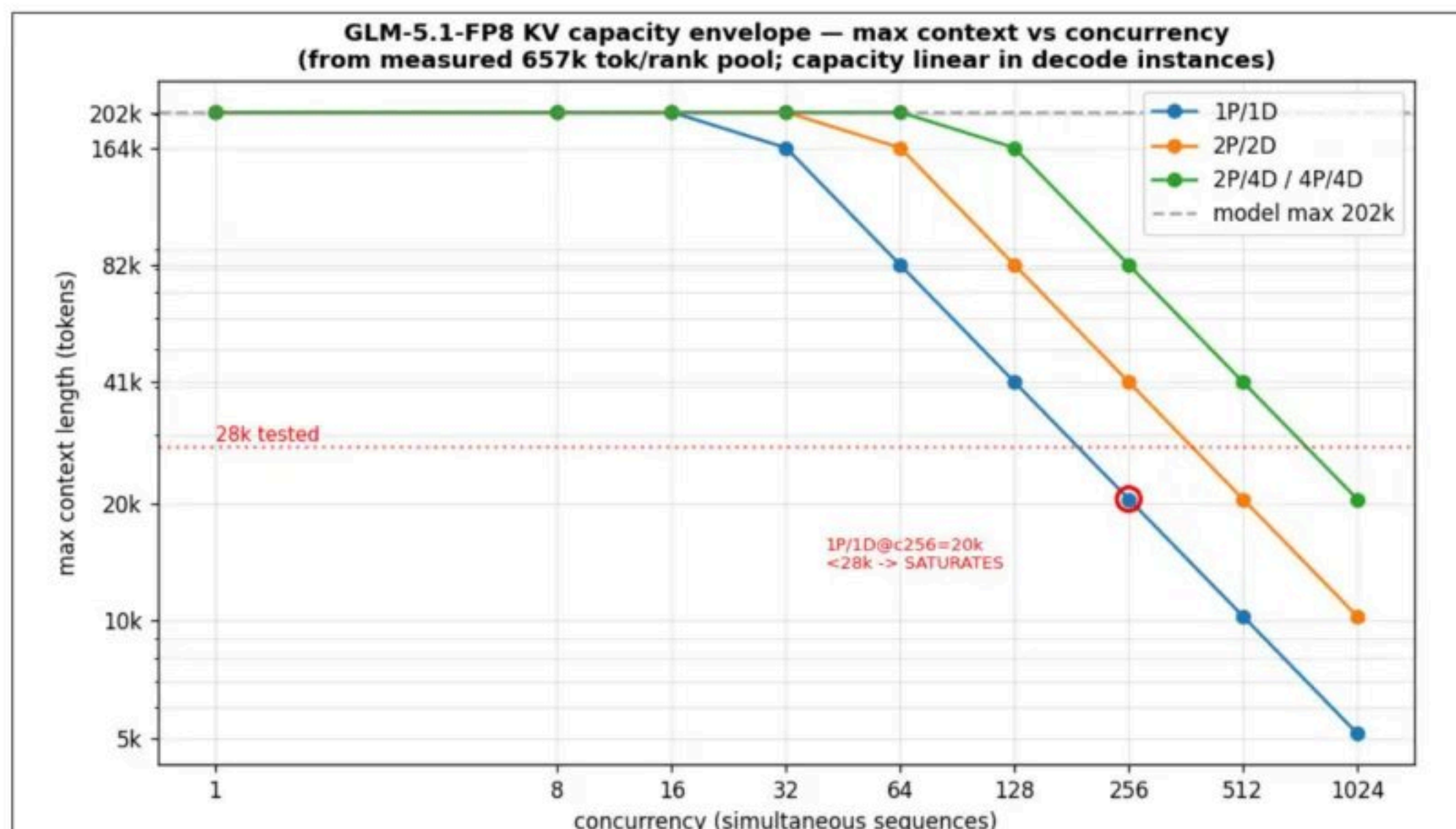


Fig 1 — KV capacity envelope: max context vs. concurrency by topology [1]

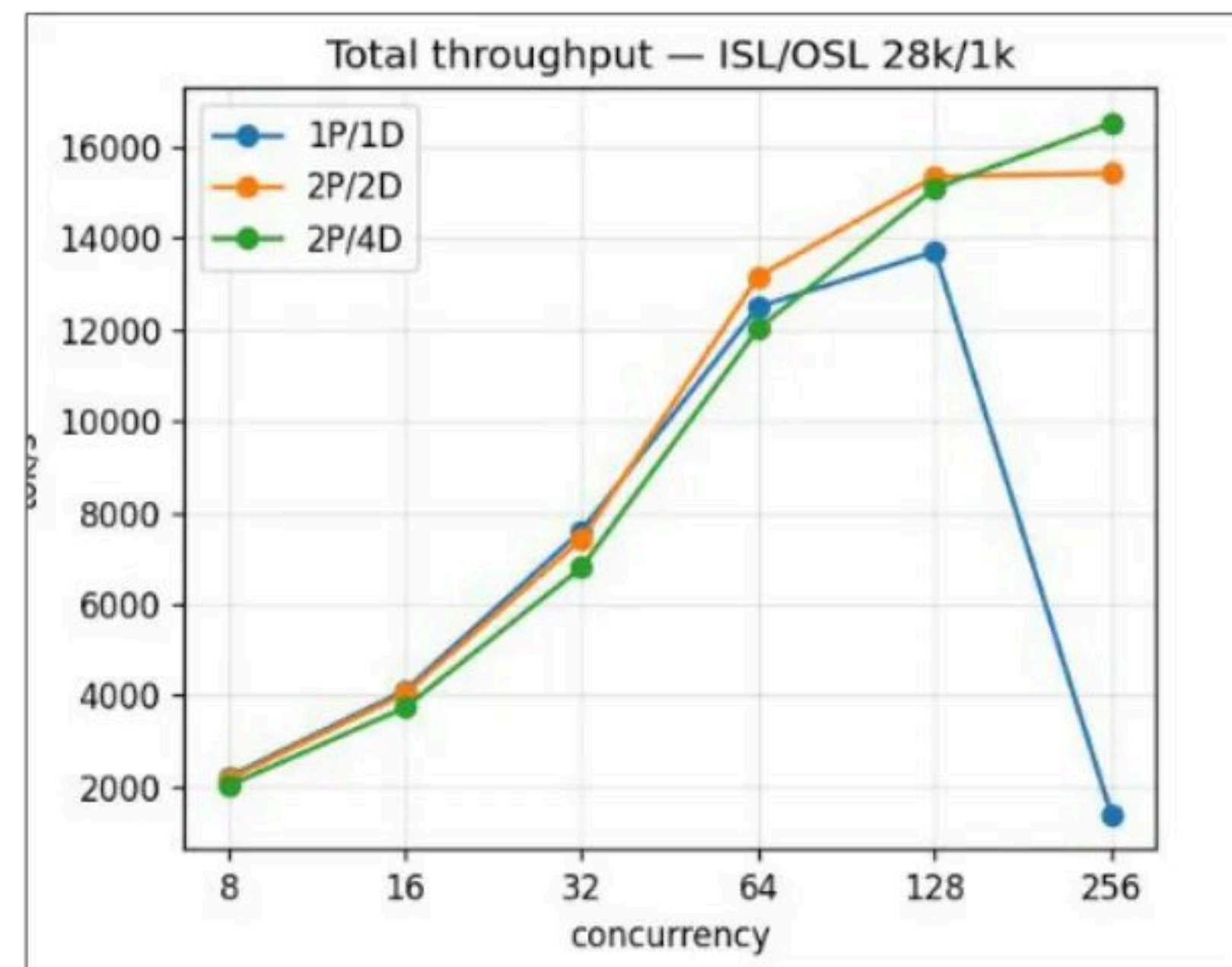


Fig 2 — Decode throughput at 28k/1k as concurrency rises [1]

The drop is not a mystery — it is arithmetic. vLLM reports the measured per-rank KV pool (657,000 tokens per decode rank), and GLM-5.1's per-token KV cost is fixed: MLA latent (576 B) plus the DSA indexer (128 B), across 78 layers, is 53.6 KiB per token in FP8. So the longest context a topology can keep resident for all in-flight sequences has a closed form:

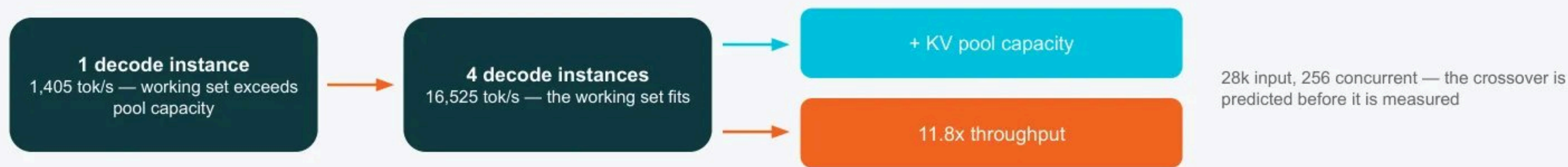
$$\begin{aligned} \text{max_context} &= 657,000 \times 8 \times (\text{decode instances}) / \text{concurrency} \quad \text{tokens, full pool} \\ &= 5.26\text{M} \times (\text{decode instances}) / \text{concurrency} \quad \text{capped at the model's 202k limit} \end{aligned}$$

[1] Fig 1 and Fig 2 — ROCm blog, "DI Series: Scaling GLM-5.1-FP8 to 64 MI300X GPUs" — rocm.blogs.amd.com/software-tools-optimization/di-glm-wideep

Sizing the decode pool by KV capacity

- **At 28k input and 256 concurrent requests**, a single decode instance sustains **1,405 tok/s**
- The driver arithmetic: the KV working set is 7.34M tokens; one decode pool holds 5.26M
- **Adding decode instances recovers the order of magnitude: 2P/4D peaks at 16,525 tok/s on the same shape**
- **A closed-form capacity model** matches where the crossover appears, and predicts each topology's long-context reach
- The rule: size the decode pool by KV capacity, not by FLOPs — long context is a memory problem
- Validated against measurement across all four topologies in the 72-cell matrix [1]

ONE DECODE INSTANCE VS. A DECODE POOL



[1] ROCm blog, "DI Series: Scaling GLM-5.1-FP8 to 64 MI300X GPUs" — rocm.blogs.amd.com/software-tools-optimization/di-glm-wideep

Configuration: 8× AMD Instinct™ MI300X GPU (gfx942) per node, RoCEv2 fabric, eight GPU-local backend NICs; vLLM built from source (wide-EP WRITE mode + PR #47766), AITER e03fa6040, MoRI 42e895472b08; GLM-5.1-FP8. Benchmark results are preliminary; actual performance may vary based on configuration, software versions, drivers, and optimizations.

vLLM DI CI — what the nightly matrix covers

- An upstream DI CI pipeline for ROCm — disaggregated prefill/decode over the MoRI-IO KV connector & MoRI EP, on Buildkite plus Slurm
- RFC #48117 sets the plan and the test matrix; PR #47030 is the initial implementation, now merged [1]
- Gated on end-to-end accuracy, not just exit codes — GSM8K run per topology
- The multi-node RDMA path gets automated nightly coverage
- Every test group lands as a row on a public status dashboard, refreshed nightly

FOUR AXES OF THE MATRIX

Topologies

- 1P1D and 2P2D splits
- Single-node and multi-node
- vLLM Router in the loop

Parallelism

- TP8 and DP8/EP8 controls
- Wide EP DP16/EP16
- Spanning hosts

Hardware & Fabric

- MI355X node class
- AMD AI NIC fabric
- 8 NICs per node, one per GPU

Models

- DeepSeek V3, DSR1-MXFP4
- GLM-5.2-FP8, Kimi-K2.5-MXFP4
- MiniMax-M3-FP8

one pipeline, four axes — a test group is a point in this matrix, and the dashboard is the matrix rendered nightly

[1] vLLM RFC #48117 and PR #47030 — github.com/vllm-project/vllm — and the DI CI pipeline, buildkite.com/vllm/amd-distributed-inference-ci

Where the matrix stands today — vLLM DI CI

- **The controls are clean** — TP8 and DP8/EP8 disaggregated runs pass consistently on ROCm
- EP8/DP8 intranode sits at 30/30 passing [1]
- DP16 internode is validated across four models and PRs in review
- **Wide TP** coverage is in the matrix
- **WideEP DP16/EP16**: under active development
- Nightly results are triaged and owned, with upstream issues tracked
- MoRI upgrade and API changes landed in vLLM #56073, with router support for remote TP size

HOW A NIGHTLY RESULT IS TRIAGED

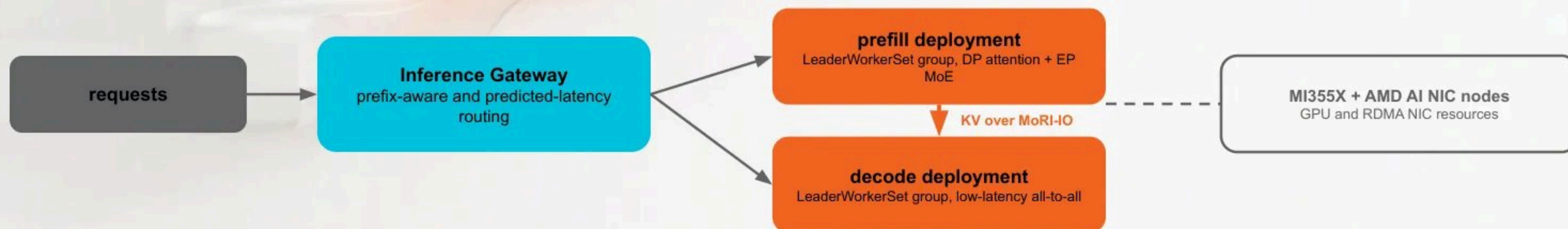


[1] vLLM AMD distributed inference CI — buildkite.com/vllm/amd-distributed-inference-ci/builds/60

llm-d Wide EP with MoRI on ROCm

- Wide EP now supported on llm-d end-to-end workflow on ROCm — LeaderWorkerSet, 2P2D
- The enablers are merged: vLLM #45043 and llm-d router #2094, with deployment guides in #1735 [1]
- MoRI sits underneath — MoRI-IO 1P1D added as a new test, MoRI-EP and MoRI-IO are covered by the nightly matrix
- Both routing paths are covered nightly — precise prefix cache and predicted latency routing both pass

WIDE EP ON llm-d — NIGHTLY LWS 2P2D ON ROCm NODES



the Wide EP recipe you would hand-launch is now a LeaderWorkerSet the nightly workflow brings up, benchmarks and tears down — on the same vLLM ROCm image

[1] vLLM PR #45043 — github.com/vllm-project/vllm — and llm-d router PR #2094, deployment guides PR #1735, nightly ROCm e2e workflows — github.com/llm-d/llm-d

Where the matrix stands today — llm-d on ROCm

- Wide EP 2P2D passes nightly
- KV-cache offloading is landing end to end (PR #2464), so capacity stops being bounded by HBM [1]
- Prefix cache and predicted latency routing are passing
- Each track lands as a well-lit path plus a nightly workflow, not a one-off recipe

FOUR PUBLIC WORKSTREAMS

Wide EP

- Beyond LWS 2P2D (wide-ep-lws guide)
- Workload autoscaling — PR #2469

KV capacity

- KV offloading end to end — PR #2464
- Tiered cache past HBM (tiered-prefix-cache)
- Longer context, same footprint

Smart routing

- Prefix cache aware today
- Predicted latency routing guide

Fabric

- RDMA fabric with GPU-direct support
- Covered by the nightly matrix
- MoRI-IO via the llm-d router

Each path ships with a nightly workflow you can run

[1] llm-d well-lit paths and nightly ROCm end-to-end workflows — github.com/llm-d/llm-d — KV offloading PR #2464, autoscaling PR #2469

Resources

Blogs & documentation

Disaggregated inference with GLM + Wide-EP

rocm.blogs.amd.com/software-tools-optimization/di-glm-wideep

Wide expert parallelism for DeepSeek

rocm.blogs.amd.com/software-tools-optimization/wide-ep-deepseek

vLLM + MoRI recipe (ROCm docs)

rocm.docs.amd.com/projects/ai-ecosystem/.../vllm-mori-recipe.html

MoRI-IO KV connector for vLLM

vllm.ai/blog/2026-04-07-morrio-kv-connector

Prefill / decode disaggregation on ROCm

rocm.blogs.amd.com/software-tools-optimization/disaggregation

End-to-end and fine-grained tracing in llm-d

llm-d.ai/blog/end-to-end-and-fine-grained-tracing-in-llm-d

Quick DI recipes on AMD GPUs (ROCm/MAD)

SGLang disaggregated serving script

github.com/ROCm/MAD/tree/develop/scripts/sglang_disagg

vLLM disaggregated serving script

github.com/ROCm/MAD/tree/develop/scripts/vllm_disagg

Where to start: read the disaggregation blog for the concepts, then run the MAD scripts to reproduce a working prefill/decode setup end to end.

Developer resources

Get hands on with AMD Instinct GPUs — scan a code or use the link below it.

AMD Developer Cloud



Spin up MI300X instances in minutes — \$100 in complimentary credits for 30 days, no local hardware needed.

www.amd.com/en/developer/resources/cloud-access/amd-developer-cloud.html

AMD AI Developer Program



Free, self-paced AI Academy learning paths, tutorials and developer blogs for building on ROCm.

developer.amd.com/amd-ai-academy/

COPYRIGHT AND DISCLAIMER

©2026 Advanced Micro Devices, Inc. All rights reserved.

AMD, the AMD Arrow logo, AMD AI NIC™ adapter, AMD Instinct™ GPU, ROCm™ software and combinations thereof are trademarks of Advanced Micro Devices, Inc. Other product names used in this publication are for identification purposes only and may be trademarks of their respective companies.

The information presented in this document is for informational purposes only and may contain technical inaccuracies, omissions, and typographical errors. The information contained herein is subject to change and may be rendered inaccurate for many reasons, including but not limited to product and roadmap changes, component and motherboard version changes, new model and/or product differences between differing manufacturers, software changes, BIOS flashes, firmware upgrades, or the like. Any computer system has risks of security vulnerabilities that cannot be completely prevented or mitigated. AMD assumes no obligation to update or otherwise correct or revise this information. However, AMD reserves the right to revise this information and to make changes from time to time to the content hereof without obligation of AMD to notify any person of such revisions or changes.

THIS INFORMATION IS PROVIDED "AS IS." AMD MAKES NO REPRESENTATIONS OR WARRANTIES WITH RESPECT TO THE CONTENTS HEREOF AND ASSUMES NO RESPONSIBILITY FOR ANY INACCURACIES, ERRORS, OR OMISSIONS THAT MAY APPEAR IN THIS INFORMATION. AMD SPECIFICALLY DISCLAIMS ANY IMPLIED WARRANTIES OF NON-INFRINGEMENT, MERCHANTABILITY, OR FITNESS FOR ANY PARTICULAR PURPOSE. IN NO EVENT WILL AMD BE LIABLE TO ANY PERSON FOR ANY RELIANCE, DIRECT, INDIRECT, SPECIAL, OR OTHER CONSEQUENTIAL DAMAGES ARISING FROM THE USE OF ANY INFORMATION CONTAINED HEREIN, EVEN IF AMD IS EXPRESSLY ADVISED OF THE POSSIBILITY OF SUCH DAMAGES.

3:30–4:00 PM

Coffee & Snacks

Sessions resume at 4:00 PM



SESSION

Inside vLLM Semantic Router

Intelligent, cache-aware routing based on request semantics, model capabilities, workload characteristics, and infrastructure constraints.

Aayush Saini

Red Hat

<https://aadeshai.xyz/presentation-vllm-sr/>

Thank you!